# STM32基础 - USART串口通信-语音控制

## 课程目标

1. 熟悉串口通信协议
2. 熟悉USART基本结构
3. 掌握USART串口通信的工作过程

## 课程试验

1. 利用STM32完成串口数据发送和接收试验
2. 利用STM32完成串口收发Hex数据包试验
3. 利用STM32完成串口收发文本数据包试验

## 课堂引入

USART因为其通信速率、距离、硬件特性等已经不适合PC的要求，在现代的通用计算机上几乎消失殆尽了，取而代之的是USB口。但是在嵌入式应用领域里，USART的地位仍是无可替代的，在嵌入式硬件平台上，对通信的数据量、速率等要求并不是很高，而USART极低的硬件资源消耗、不错的可靠性、简洁的协议以及高度的灵活性，使得其非常符合嵌入式设备的应用需求。利用USART可以轻松实现PC与嵌入式主控器的通信，如用以实时查看一些变量的值，这对于调试手段受到局限的嵌入式平台显得非常重要。可以这样说，一个电子开发人员，拿到任何一款单片机，所要做的第一件事是熟悉它的开发环境，而第二件事就是要会使用它的USART。



## 授课进程

## 一、通信接口

### 1、基本介绍

通信的目的是将一个设备的数据传送到另一个设备，扩展硬件系统。比如，STM32芯片集成了很多功能模块，定时器计数、PWM输出、AD采集等等，这些都是芯片内部的电路。这些电路的配置寄存器、数据寄存器都在芯片里面，操作这些寄存器非常简单，直接读写就行了。但是也有些功能是STM32内部所没有的，比如想要蓝牙无线遥控的功能，想要陀螺仪加速度计测量姿态的功能，STM32没有，就需要通过外挂芯片来完成。外挂的芯片的数据都在STM32外面，STM32如要获取到这些数据就需要在这两个设备之间，连接上一根或多根通信线，通过通信线路发送或者接收数据完成数据交换，从而实现控制外挂模块和读取外挂模块数据的目的。

通信协议：制定通信的规则，通信双方按照协议规则进行数据收发。通信的目的是进行信息传递，双方约定的规则就是通信协议。

| 名称 | 引脚 | 常用在 | 双工 | 时钟 | 电平 | 设备 |
| --- | --- | --- | --- | --- | --- | --- |
| USART | TX、RX | 单片机和PC,单片机和单片机 | 全双工 | 异步 | 单端 | 点对点 |
| I2C | SCL、SDA | 单片机和单片机 | 半双工 | 同步 | 单端 | 多设备 |
| SPI | SCLK、MOSI、MISO、CS | 单片机和单片机 | 全双工 | 同步 | 单端 | 多设备 |
| CAN | CAN_H、CAN_L | 智能汽车 | 半双工 | 异步 | 差分 | 多设备 |
| USB | DP、DM | | 半双工 | 异步 | 差分 | 点对点 |

- 单工：单向传输，仅有一个数据流方向。
- 半双工：双向传输，但不能同时进行，需交替使用通信链路。
- 全双工：双向同时传输，数据可在两个方向上独立、并发地流动。
- 单端信号：使用单一信号线和一个参考地来表示逻辑状态，
- 而差分信号：利用一对信号线的电压差来传输信息，具有更强的抗干扰能力和更高的信号质量，尤其适用于需要高可靠性和高性能的通信环境。

### 2、串口通信

串口是一种应用十分广泛的通讯接口，串口成本低、容易使用、通信线路简单，可实现两个设备的互相通信。单片机的串口可以使单片机与单片机、单片机与电脑、单片机与各式各样的模块互相通信，极大地扩展了单片机的应用范围，增强了单片机系统的硬件实力。

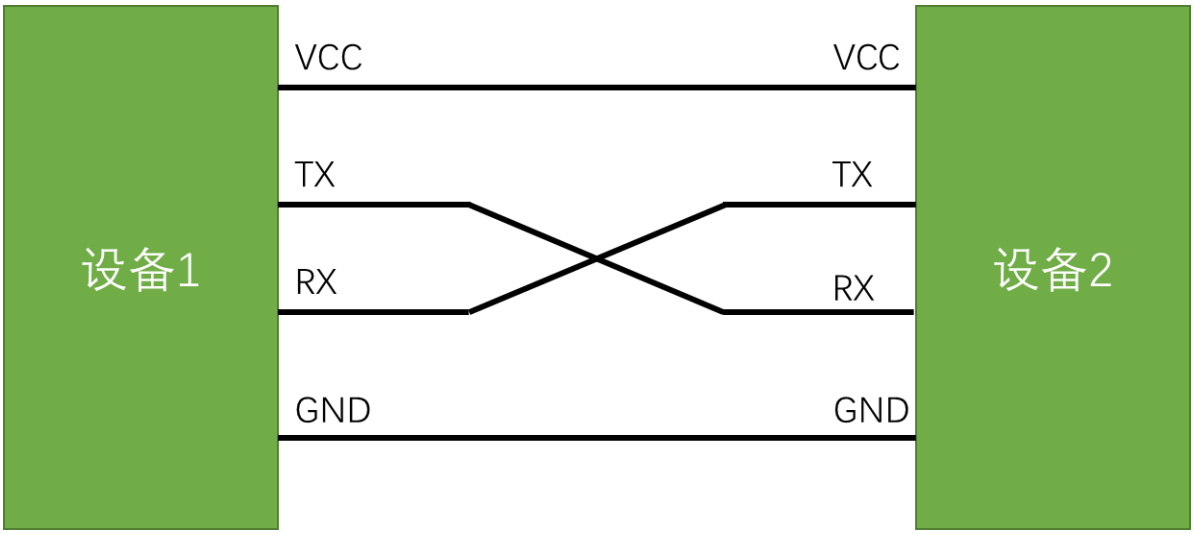


左边是USB转串口模块，芯片型号是CH340，这个芯片可以把串口协议转换为USB协议。它一边是USB口，可以插在电脑上，另一边是串口的引脚，可以和支持串口的芯片接在一起，这样就能实现串口和电脑的通信了。中间是一个陀螺仪传感器的模块，可以测量角速度、加速度这些姿态参数，它左右各有四个引脚，一边是串口的引脚，另一边是I2C的引脚。右边是蓝牙串口模块，下面四个角是串口通信的引脚，上面的芯片可以和手机互联，实现手机遥控单片机的功能。

**3、硬件电路**

简单双向串口通信有两根通信线（发送端TX和接收端RX），TX与RX需要交叉连接。当只需单向的数据传输时，可以只接一根通信线。

串口是有很多电平标准的，像直接从控制器里出来的信号，一般都是TTL电平，相同的电平才能互相通信。如果是不同的电平信号，则
需要加一个电平转换芯片转接一下。




**4、电平标准**

电平标准是数据1和数据0的表达方式，是传输线缆中人为规定的电压与数据的对应关系，串口常用的电平标准有如下三种：

（1）TTL电平：

+3.3V或+5V表示1，0V表示0，这是一种最为常见的电平信号。

（2）RS232电平：-3~-15V表示1，+3~+15V表示0。

一般在大型的机器上适用，由于环境可能比较恶劣，静电干扰比较大，所以电平的电压都比较大，而且允许波动的范围也很大。

（3）RS485电平：

两线压差+2~+6V表示1，-2~-6V表示0（差分信号）。差分信号抗干扰能力非常强，使用RS485电平标准，通信距离可以达到上千米。而前面两种电平，最远只能达到几十米，再远就传不了了。

由于单片机属于低压小型设备，所以后续讲解使用的都是TTL电平。如果要做的设备需要其它的电平，就再加电平转换芯片就行了，因为在软件层面上，它们都属于串口，故而程序上并不会有什么区别。

**5、串口参数及时序**

**波特率**：串口通信的速率( baud)，多长时间取一次电平数据。

- 如果一个码元仅对应一个比特，则波特率直接等于比特率，即每秒钟传输的比特数
- 多长时间取一次数据，波特率越高传输越快，接收和发送要求波特率一样

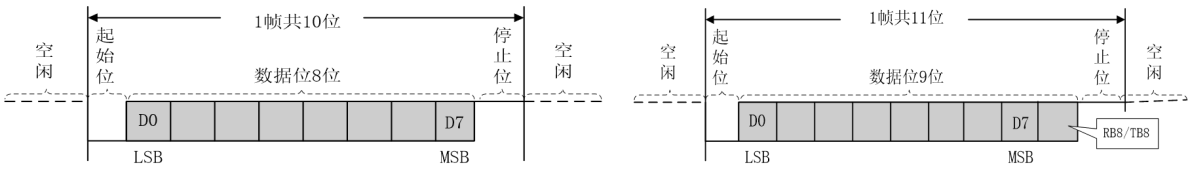
**起始位**：标志一个数据帧的开始，**固定为低电平。**

**数据位**：数据帧的有效载荷，1为高电平，0为低电平，**低位先行。**

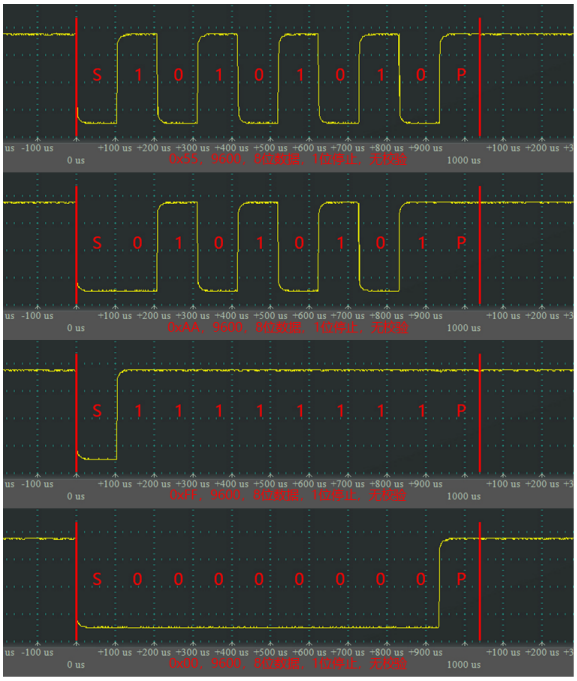
**校验位**：用于数据验证，根据数据位计算得来。

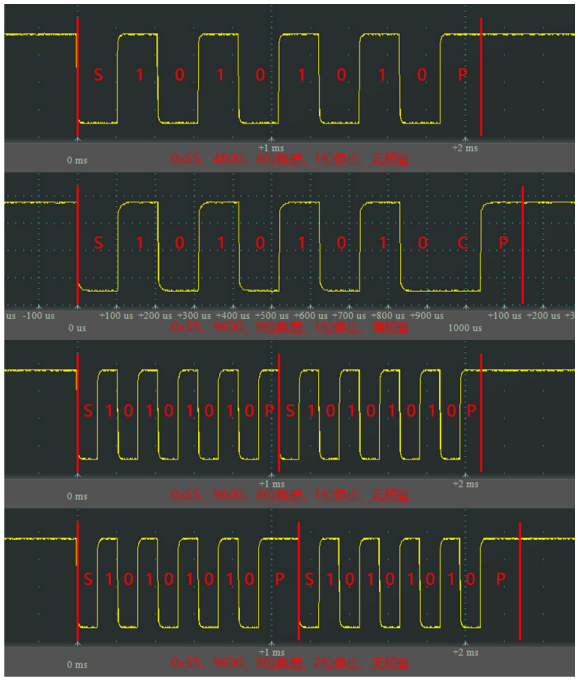
- **无校验（None）**：
  - **规则**：不添加校验位，数据帧仅包含起始位、数据位（通常为5、6、7、8位）和停止位。这是最简单的一种模式，不需要对数据进行额外的计算或检查。
- **奇校验（Odd Parity）**：
  - **规则**：在数据位之后附加一位奇校验位，使得整个数据帧（包括数据位和校验位）中“1”的个数为奇数。如果数据位中“1”的个数已经是奇数，则校验位为“0”，以保持奇数个“1”的总数；反之，若数据位中“1”的个数为偶数，则校验位为“1”，以使总“1”数变为奇数。
- **偶校验（Even Parity）**：
  - **规则**：同样在数据位之后附加一位偶校验位，使得整个数据帧中“1”的个数为偶数。如果数据位中“1”的个数已经是偶数，则校验位为“0”，保持偶数个“1”的总数；反之，若数据位中“1”的个数为奇数，则校验位为“1”，以使总“1”数变为偶数。

**停止位**：用于数据帧间隔，固定为**高电平。**




**6、串口时序**




图示是利用示波器实测的串口波形图，操作方法是把示波器的一个探头的GND接在负极，另一个探头接在发送设备的TX引脚，然后发送数据就能获得波形了。第一个波形是发送一个字节数据0x55时，在TX引脚输出的波形。波特率是9600，所以每位的时间就是1/9600，大概是104微秒。可以看到图中一位就是100微秒多一点，就是104微秒。没发送数据的时候是空闲状态高电平，数据帧开始，先发送起始位，产生下降沿，代表数据帧开始。数据0x55转为二进制，第一位先行就是依次发送1010 1010。这个参数是8位数据，1位停止，无校验。没有校验位，所以后面就是停止位，把引脚置回高电平，这样一个数据帧就完成了。

在STM32中这个根据自己数据翻转高低电平是由USART外设自动完成的，不用我们操心。当然也可以利用软件模拟产生这样的波形。就是定时器定一个104微秒的时间，时间到之后，按照数据帧的要求，调用GPIO_WriteBit置高低电平，产生一个和上面一样的波形，这样也可以完成串口通信的TX引脚发送，就是置高低电平。在RX引脚接收，就是读取高低电平，这也可以由USART外设自动来完成的，不用我们操心。如果想要软件模拟的话，那就是定时调用GPIO_ReadInputDataBit来读取每一位，最终拼接成一个字节。当然接收的时候，应该还需要一个外部中断，在起始位的下降沿触发进入接收状态，并且对齐采样时钟，然后依次采样8次，这就是接收的逻辑。

接着看下面的波形，如果发送0xAA，波形就是第二行左边的样子。起始位、然后01010101、停止位、结束。

第三行左边的波形是发送0xFF，就是8个1，波形就是起始位、8个1、停止位、结束。在起始位下降沿之后的一个数据帧的时间内，这个高电平就是作为数据1来看的，当数据帧结束后，这里虽然还是1没有任何变化，但此时的1已经是属于空闲状态了。它需要等待下一个下降沿，来开启新的一帧数据。

再看最下面的波形，如果发送0x00，就是8个0，那波形就是起始位、8个0、停止位置回高电平这样来进行。

继续看右边的波形，如果把波特率改成4800呢？也就是波特率变为一半，那相应的波形时长就会变为原来的2倍。可以看到这里10位数据，总共大概2毫秒多一点，具体应该在2.08毫秒。那一位就是208微秒，是之前的2倍。数据波形的时间拉宽，波形的变化趋势是不变的。

接着看第二行的波形，最后加了一个偶校验位，数据是0x55。数下1的个数是4个，已经是偶数了，所以输出的校验位是0，低电平，此时包括校验位在内的数据总共是偶数个1，总的波形就是这个样子。

最后看一下停止位的变化

串口的停止位是可以进行配置的，可以选择1位、1.5位、2位等。在第三行的波形上是1位停止位，连续发送两个0x55，两个数据帧会接在一起，中间没有空闲状态。最后一行右边的波形是2位停止位，连续发送两个0x55，可以看到其停止位就是两位的宽度，中间也没有空闲状态，不过这样数据分隔的就更宽一些了。

总结一下就是TX引脚输出定时翻转的高低电平，RX引脚定时读取引脚的高低电平，每个字节的数据加上起始位、停止位、可选的校验位

打包成数据帧，依次输出在TX引脚，另一端RX引脚依次接收，这样就完成了字节数据的传递，这就是串口通信。

## 二、USART

### 1、基本介绍

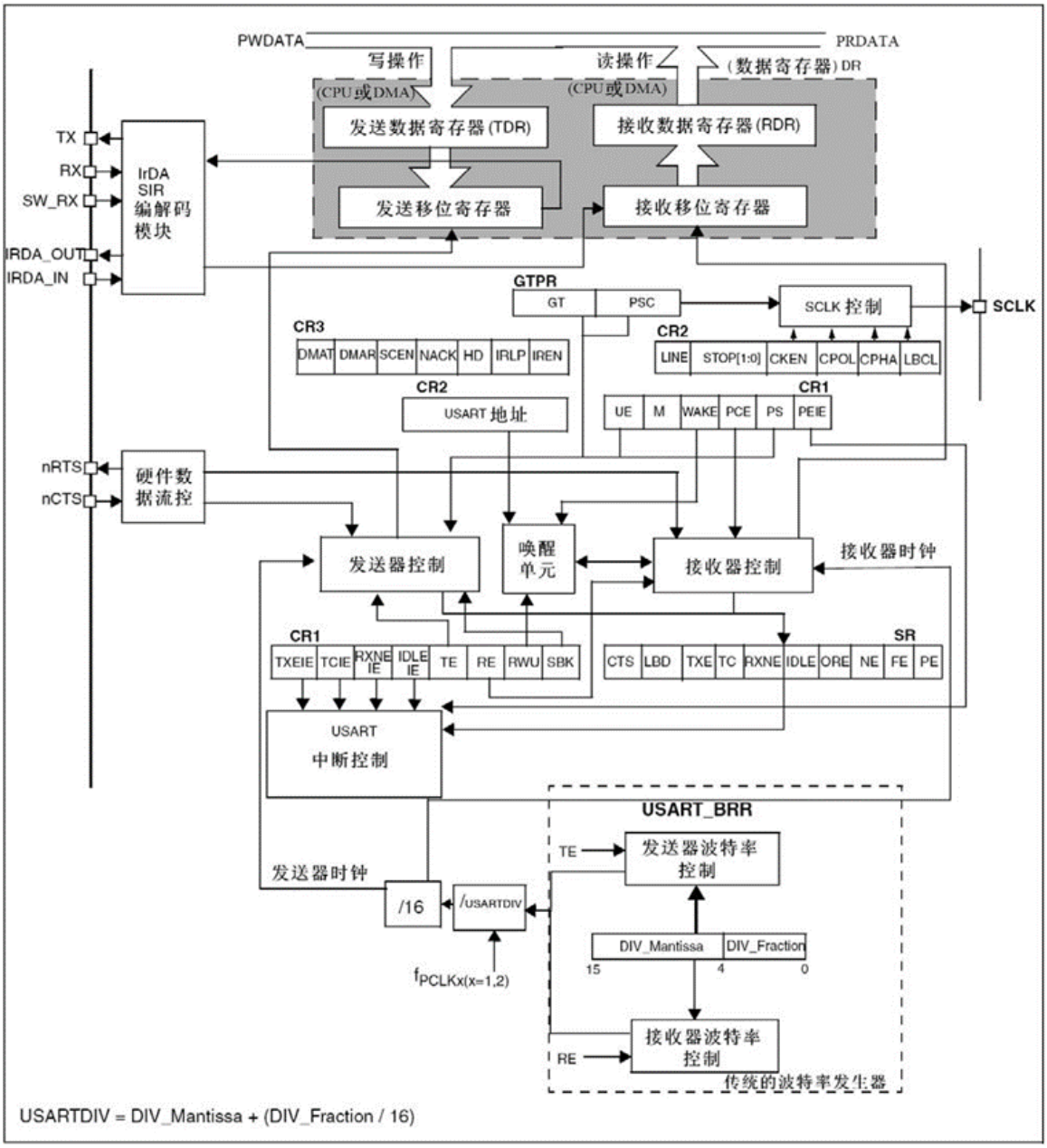
USART（Universal Synchronous/Asynchronous Receiver/Transmitter）通用同步/异步收发器。这里S就是同步的意思，有时候还会见到UART的写法，少了一个S就是异步收发器。一般串口很少用到同步的功能，所以USART和UART通常也没有太大的区别。STM32的时钟同步只是时钟的输出，目的是为了兼容其他协议，并不支持两个USART外设之间进行同步通信，所以我们还是以异步通信的学习为主。

USART是STM32内部集成的硬件外设，可根据数据寄存器的一个字节数据自动生成数据帧时序，从TX引脚发送出去，也可自动接收RX引脚的数据帧时序，拼接为一个字节数据，存放在数据寄存器里。自带波特率发生器，最高达4.5Mbits/s，可配置数据位长度（8/9）、停止位长度（0.5/1/1.5/2），可选校验位（无校验/奇校验/偶校验），支持同步模式、硬件流控制、DMA、智能卡、IrDA、LIN局域网通信等。

STM32F103C8T6芯片的USART资源包含**USART1**、 **USART2**、 **USART3**。

### 2、USART框图

图248　USART框图




TX：发送数据输出引脚。

RX：接收数据输入引脚。

SW_RX：数据接收引脚，只用于单线和智能卡模式，属于内部引脚，没有具体外部引脚。

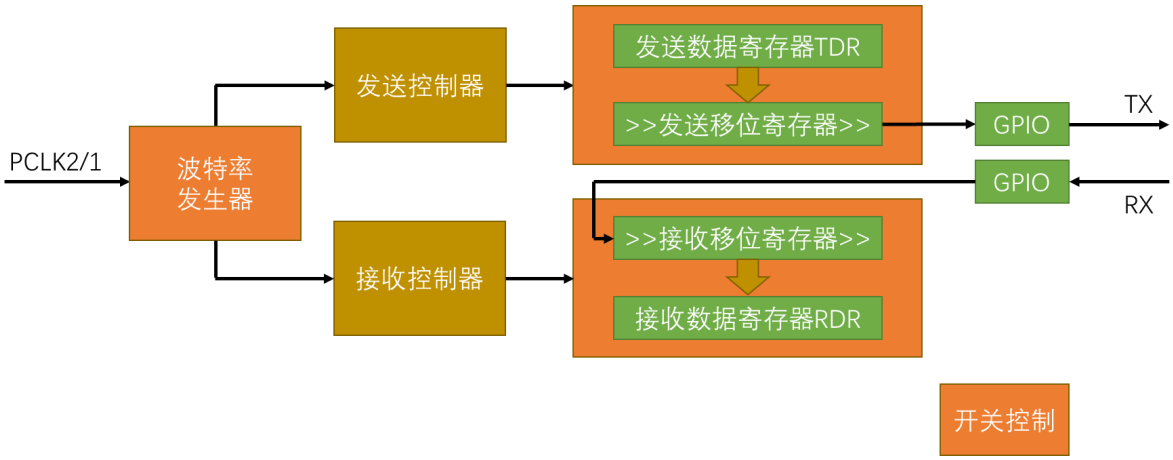
IrDA_RDI: IrDA模式下的数据输入。
IrDA_TDO: IrDA模式下的数据输出。

nRTS：请求以发送（Request To Send），n 表示低电平有效。如果使能RTS流控制，当 USART 接收器准备好接收新数据时就会将nRTS变成低电平；当接收寄存器已满时，nRTS 将被设置为高电平。该引脚只适用于硬件流控制。

nCTS：清除以发送（Clear To Send），n 表示低电平有效。如果使能CTS流控制，发送器在发送下一帧数据之前会检测nCTS引脚，如果为低电平，表示可以发送数据，如果为高电平则在发送完当前数据帧之后停止发送。该引脚只适用于硬件流控制。

SCLK：发送器时钟输出引脚。这个引脚仅适用于同步模式。

### 3、基本结构




USART基本结构图。左边是波特率发生器，用于产生约定的通信速率。时钟来源是PCLK2/1，经过波特率发生器分频后产生的时钟通向发送控制器和接收控制器。发送控制器和接收控制器用来控制发送移位和接收移位。之后由发送数据寄存器和发送移位寄存器这两个寄存器的配合将数据一位一位的移出去。通过GPIO口的复用输出，输出到TX引脚，产生串口协议规定的波形。其中右移的符号代表移位寄存器是往右移的，是低位先行。当数据由数据寄存器转移到移位寄存器时，会置一个TXE的标志位，通过判断这个标志位，就可以知道是不是可以写下个数据了。接收部分也是类似的。RX引脚的波形，通过GPIO口输入，在接收控制器的控制下，一位一位的移入接收移位寄存器。接收移位寄存器也是右移的，因为是低位先行，所以要从左边开始移进来，移完一帧数据后，数据就会统一转运到接收数据寄存器。在转移的同时，置一个RXNE标志位，通过检查这个标志位，就可以知道是不是收到数据了。同时这个标志位也可以去申请中断，这样就可以在收到数据时，直接进入中断函数，然后快速的读取和保存数据。图中实际上有4个寄存器，但是在软件层面，只有一个DR寄存器可以用于读写。写入DR时，数据走发送这条路，进行发送。读取DR时，数据走接收这条路，进行接收。这就是USART进行串口数据收发的过程。在上图右下角有一个开关控制，这个就是配置完成之后，用CMD开启外设，这算是常规操作了。

### 4、数据帧

图249　字长设置

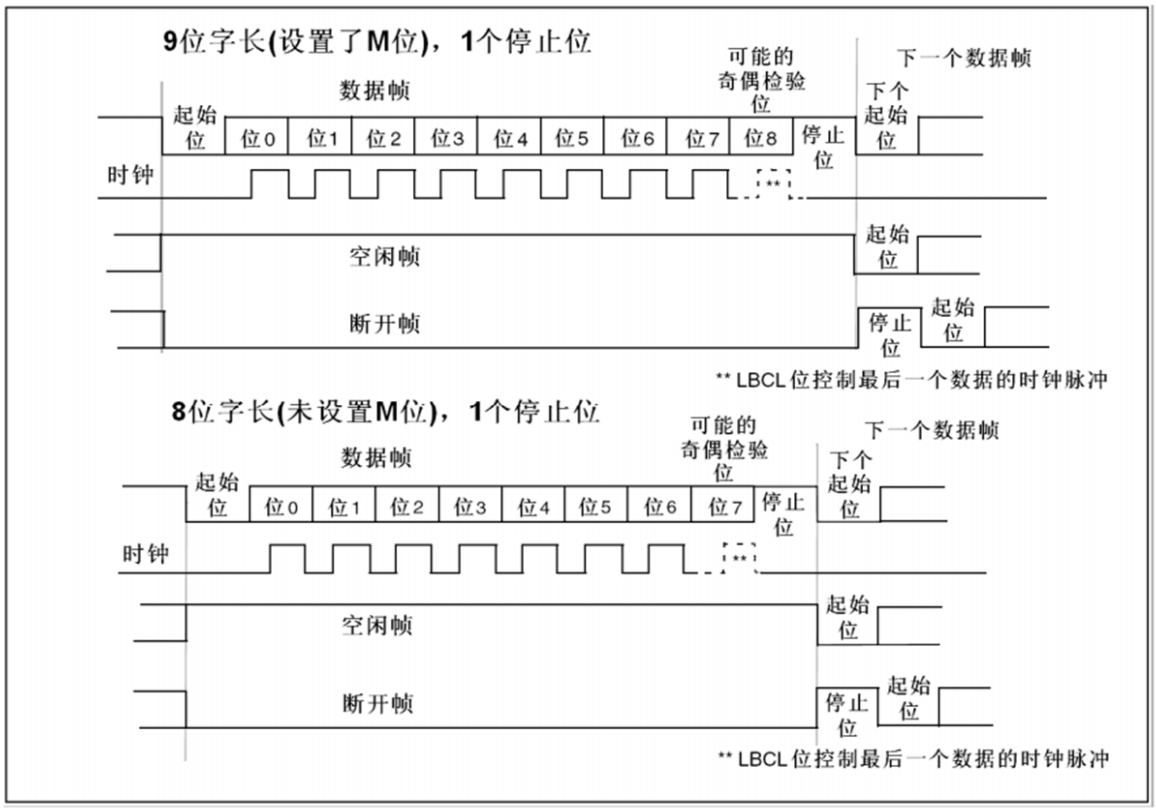
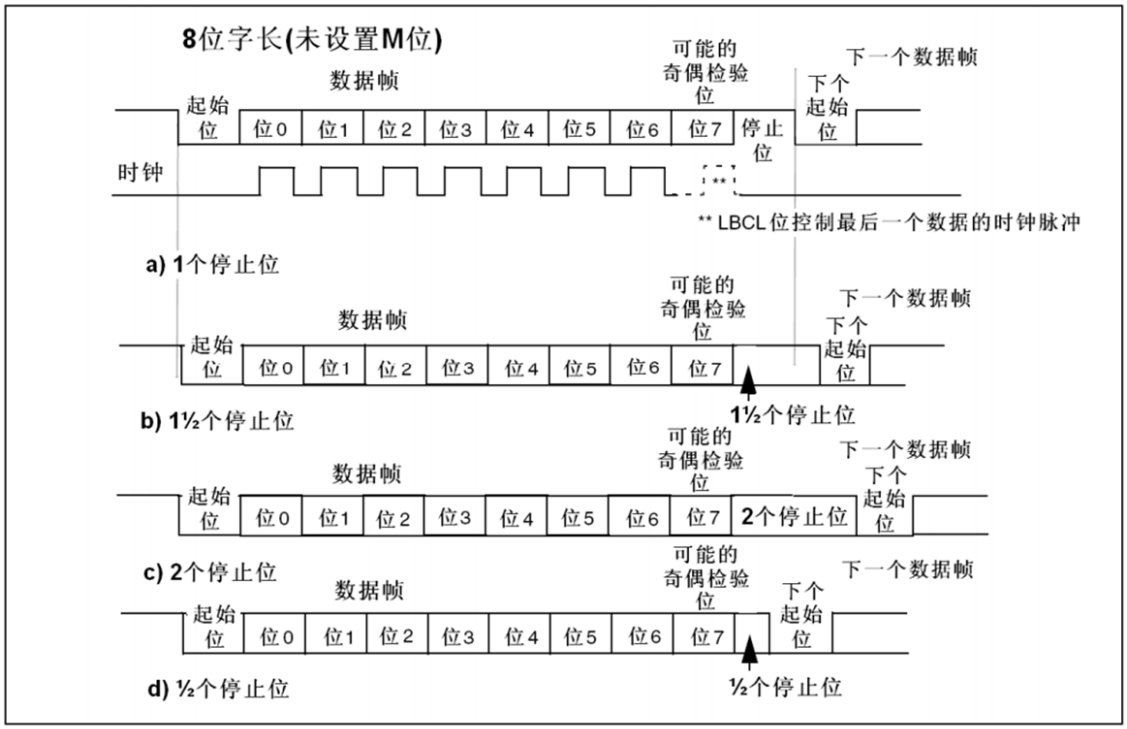
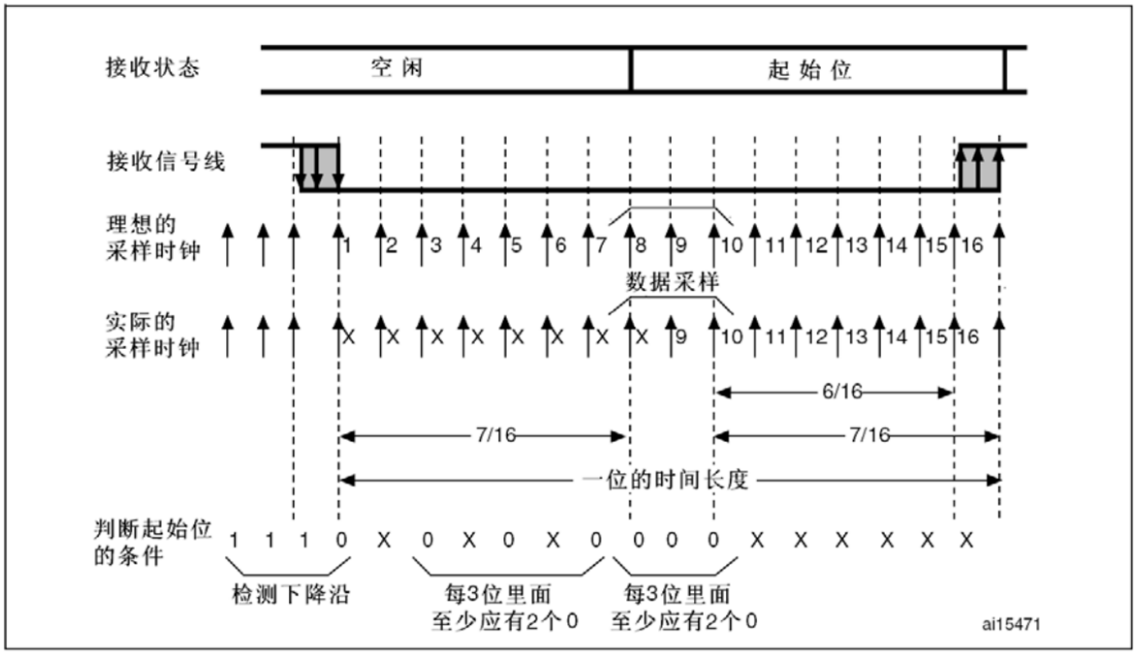


图250　配置停止位



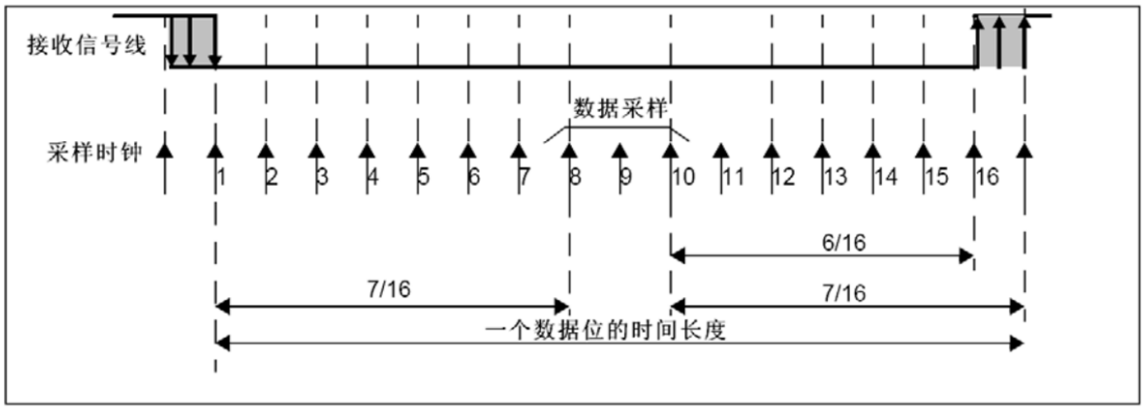

## 5、起始位侦测

图252　起始位侦测




## 6、数据采样

图253　检测噪声的数据采样




**7、波特率发生器**

波特率发生器就是分频器，发送器和接收器的波特率由波特率寄存器BRR里的DIV确定。

计算公式：

$$\text{波特率} = f_{PCLK2/1}/(16 * DIV)$$

这里的PCLK2/1是给外设的时钟（USART1在APB2上一般是72MHz; USART2，3在APB1上一般为36MHz）。

表176 设置波特率时的误差计算

| 波特率 | | $f_{PCLK}$ = 36MHz | | | $f_{PCLK}$ = 72MHz | | |
|---|---|---|---|---|---|---|---|
| 序号 | Kbps | 实际 | 置于波特率寄存器中的值 | 误差% | 实际 | 置于波特率寄存器中的值 | 误差% |
| 1 | 2.4 | 2.400 | 937.5 | 0% | 2.4 | 1875 | 0% |
| 2 | 9.6 | 9.600 | 234.375 | 0% | 9.6 | 468.75 | 0% |
| 3 | 19.2 | 19.2 | 117.1875 | 0% | 19.2 | 234.375 | 0% |
| 4 | 57.6 | 57.6 | 39.0625 | 0% | 57.6 | 78.125 | 0% |
| 5 | 115.2 | 115.384 | 19.5 | 0.15% | 115.2 | 39.0625 | 0% |
| 6 | 230.4 | 230.769 | 9.75 | 0.16% | 230.769 | 19.5 | 0.16% |
| 7 | 460.8 | 461.538 | 4.875 | 0.16% | 461.538 | 9.75 | 0.16% |
| 8 | 921.6 | 923.076 | 2.4375 | 0.16% | 923.076 | 4.875 | 0.16% |
| 9 | 2250 | 2250 | 1 | 0% | 2250 | 2 | 0% |
| 10 | 4500 | 不可能 | 不可能 | 不可能 | 4500 | 1 | 0% |

## 波特比率寄存器(USART_BRR)

*注意：　如果TE或RE被分别禁止，波特计数器停止计数*

地址偏移：0x08

复位值：0x0000

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 保留 | | | | | | | | | | | | | | | |
| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
| DIV_Mantissa[11:0] | | | | | | | | | | | | DIV_Fraction[3:0] | | | |
| rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw | rw |

| | |
|---|---|
| 位31:16 | 保留位，硬件强制为0 |
| 位15:4 | **DIV_Mantissa[11:0]**：USARTDIV的整数部分<br>这12位定义了USART分频器除法因子(USARTDIV)的整数部分。 |
| 位3:0 | **DIV_Fraction[3:0]**：USARTDIV的小数部分<br>这4位定义了USART分频器除法因子(USARTDIV)的小数部分。 |

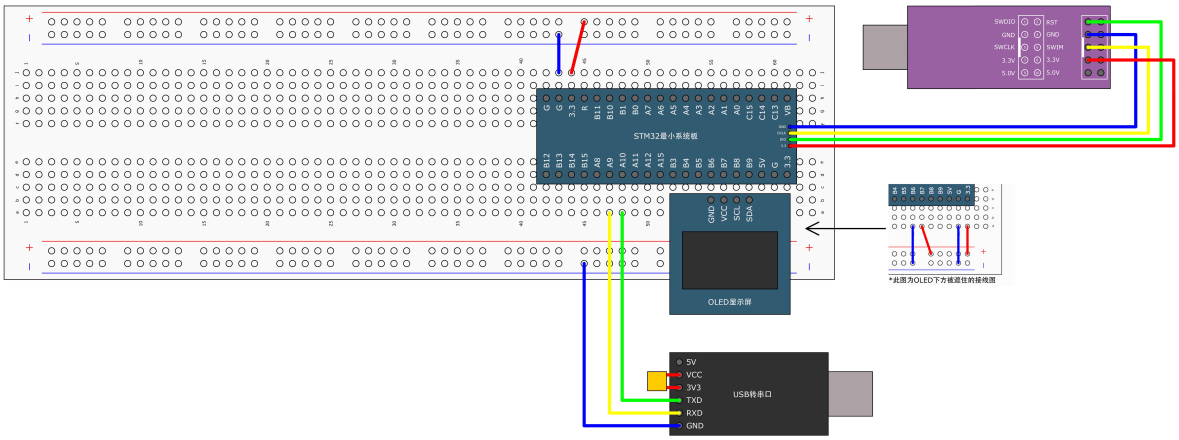
BRR寄存器里面就是分频系数DIV。DIV分为整数部分和小数部分，可以实现更细腻的分频。波特率和分频系数的关系可以由上面的计算公式进行计算，波特率等于PCLK2/1的时钟频率除以16倍的DIV。这里的16是因为它内部还有一个16倍波特率的采样时钟，所以输入时钟除以DIV要等于16倍的波特率。举个例子，比如要配置USART1为9600的波特率，该如何配置这个BRR寄存器呢？代入公式就是9600等于USART1的时钟72m除16倍的DIV等于468.75，这是一个带小数的分频系数。最终写到寄存器还需要转换成二进制，得到的二进制数是111,010,100.11，所以最终写入寄存器的就是整数部分为111010100，前面多出来的补0，小数部分为11，后面多出来的补0。

## 三、串口收发实验

（1）实验目的

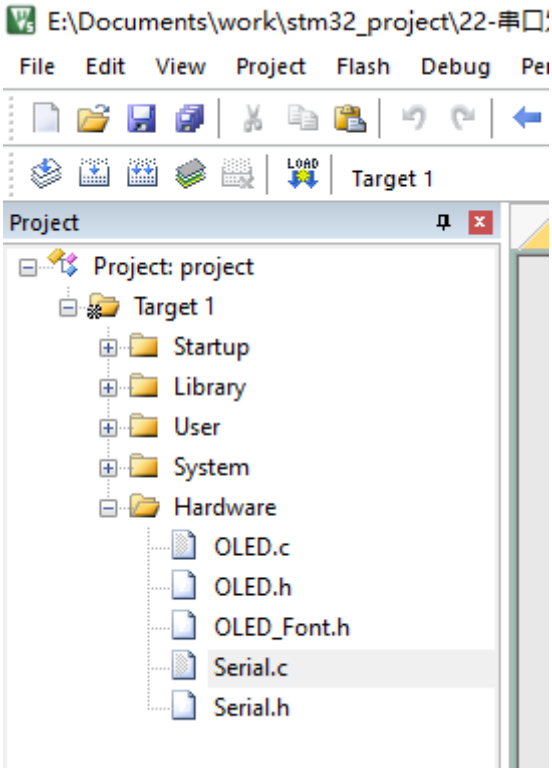
通过控制STM32的USART1实现与PC通信，可以发送数据到PC端的串口工具，也可以接收PC利用串口工具发给STM32的数据，将发送和接收的数据分别也显示在OLED上。

（2）硬件电路接线图




（3）程序设计

工程结构如下图：




其中Hardware组与之前工程相比，增加了串口驱动的代码。具体代码展示如下：

Serial.c

```c
#include "stm32f10x.h"                  // Device header
#include <stdio.h>
#include <stdarg.h>

uint8_t Serial_RxData;  // 串口接收的数据
uint8_t Serial_RxFlag;  // 串口接收的标志位

void Serial_Init(void) {
    RCC_APB2PeriphClockCmd(RCC_APB2Periph_USART1, ENABLE);  // 打开USART1的时钟
    RCC_APB2PeriphClockCmd(RCC_APB2Periph_GPIOA, ENABLE);   // 打开GPIOA的时钟

    GPIO_InitTypeDef GPIO_InitStructure;
    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_AF_PP;     // 配置复用推挽输出
    GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
    GPIO_InitStructure.GPIO_Pin = GPIO_Pin_9;
    GPIO_Init(GPIOA, &GPIO_InitStructure);

    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_IPU;       // 配置上拉输入
    GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
    GPIO_InitStructure.GPIO_Pin = GPIO_Pin_10;
    GPIO_Init(GPIOA, &GPIO_InitStructure);

    USART_InitTypeDef USART_InitStructure;
    USART_InitStructure.USART_BaudRate = 9600;  // 配置波特率
    USART_InitStructure.USART_HardwareFlowControl = USART_HardwareFlowControl_None;
// 配置不使用流控
    USART_InitStructure.USART_Mode = USART_Mode_Tx | USART_Mode_Rx;     // 配置收发模式
    USART_InitStructure.USART_Parity = USART_Parity_No;     // 配置无奇偶校验
    USART_InitStructure.USART_StopBits = USART_StopBits_1;      // 配置1位停止位
    USART_InitStructure.USART_WordLength = USART_WordLength_8b;     // 配置字长为8位
    USART_Init(USART1, &USART_InitStructure);       // 初始化USART1

    USART_ITConfig(USART1, USART_IT_RXNE, ENABLE); // 配置串口中断方式

    NVIC_PriorityGroupConfig(NVIC_PriorityGroup_2);     // 配置分组2

    NVIC_InitTypeDef NVIC_InitStructure;
    NVIC_InitStructure.NVIC_IRQChannel = USART1_IRQn;   // 配置USART1中断请求线
    NVIC_InitStructure.NVIC_IRQChannelCmd = ENABLE;     // 配置线路通道使能
    NVIC_InitStructure.NVIC_IRQChannelPreemptionPriority = 1;   // 配置抢占优先级为1
    NVIC_InitStructure.NVIC_IRQChannelSubPriority = 1;      // 配置响应优先级为1
    NVIC_Init(&NVIC_InitStructure);     // 初始化NVIC

    // USART使能
    USART_Cmd(USART1, ENABLE);
}

void Serial_SendByte(uint8_t Byte) {
    USART_SendData(USART1, Byte);   // 写入字节数据到数据寄存器
    while (USART_GetFlagStatus(USART1, USART_FLAG_TXE) == RESET);   // 等待发送完成
}
```

```c
void Serial_SendArray(uint8_t *Array, uint16_t length) {
    uint16_t i;
    for (i = 0; i < length; i++) {
        Serial_SendByte(Array[i]);      // 依次调用Serial_SendByte发送字节数据
    }
}

void Serial_SendString(char *String) {
    uint16_t i;
    // 遍历字符数组（字符串），遇到字符串结束标志位后停止
    for (i = 0; String[i] != '\0'; i++) {
        Serial_SendByte(String[i]);
    }
}

uint32_t Serial_Pow(uint32_t X, uint32_t Y) {
    uint32_t Result = 1;
    while (Y--) {
        Result *= X;    // 将X累乘到结果
    }
    return Result;  // 计算X的Y次方
}

void Serial_SendNumber(uint32_t Number, uint16_t length) {
    uint16_t i;
    // 根据数字长度遍历数字的每一位
    for (i = 0; i < length; i++) {
        // 数字需要按位取出
        Serial_SendByte(Number / Serial_Pow(10, length - i - 1) % 10 + '0');
    }
}

// 重写printf底层的fputc，重定向输出到串口
int fputc(int ch, FILE *f) {
    Serial_SendByte(ch);    // 将printf的底层重定向到自己的发送字节函数
    return ch;
}

void Serial_Printf(char *format, ...) {
    char String[100];
    va_list arg;    // 定义可变参数列表数据类型的变量arg
    va_start(arg, format);  // 从format开始，接收参数列表到arg变量
    vsprintf(String, format, arg);  // 使用vsprintf打印格式化字符串和参数列表到字符数组中
    va_end(arg);    // 结束变量arg
    Serial_SendString(String);
}

uint8_t Serial_GetRxFlag(void) {
    // 自定义串口标记位的处理
    if (Serial_RxFlag == 1) {
```

```
        Serial_RxFlag = 0;
        return 1;
    }
    return 0;
}

uint8_t Serial_GetRxData(void) {
    return Serial_RxData;   // 返回存储在变量中的接收数据
}

void USART1_IRQHandler(void) {
    // 判断是否是USART1的接收事件触发的中断
    if (USART_GetITStatus(USART1, USART_IT_RXNE) == SET) {
        Serial_RxData = USART_ReceiveData(USART1);      // 读取DR数据寄存器，存放接收的数据到变量中
        Serial_RxFlag = 1;      // 置自定义的接收标志位变量为1
        // 清除USART1的RXNE标志位。读取数据寄存器会自动清除此标志位，这里可以不用进行手动清除的。
        USART_ClearITPendingBit(USART1, USART_IT_RXNE);
    }
}
```

Serial.h

```
#ifndef __SERIAL_H
#define __SERIAL_H

#include <stdio.h>

void Serial_Init(void);
void Serial_SendByte(uint16_t Byte);
void Serial_SendArray(uint8_t *Array, uint16_t length);
void Serial_SendString(char *String);
void Serial_SendNumber(uint32_t Number, uint16_t length);
void Serial_Printf(char *format, ...);

uint8_t Serial_GetRxFlag(void);
uint8_t Serial_GetRxData(void);

#endif
```

main.c

```
#include "stm32f10x.h"                  // Device header
#include "OLED.h"
#include "Serial.h"

uint8_t RxData;
```

```c
int main(void){

    OLED_Init();
    Serial_Init();

    OLED_ShowString(1, 1, "RxData:");

    while(1){
        // 检查串口接收数据的标志位
        if (Serial_GetRxFlag() == 1) {
            RxData = Serial_GetRxData();     // 获取串口接收的数据
            Serial_SendByte(RxData);     // 串口将收到的数据回传回去，方便验证
            OLED_ShowHexNum(1, 8, RxData, 2);    // 显示收到的串口数据
        }
    }
}
```

（4）代码完成后，编译并下载到硬件中执行，会发现实验结果是通过串口收发，在PC端的串口工具中和STM32的OLED显示数据会对应变化，符合预期设计。

## 四、串口数据包

1、数据模式

HEX模式/十六进制模式/二进制模式：以原始数据的形式显示。

文本模式/字符模式：以原始数据编码后的形式显示。

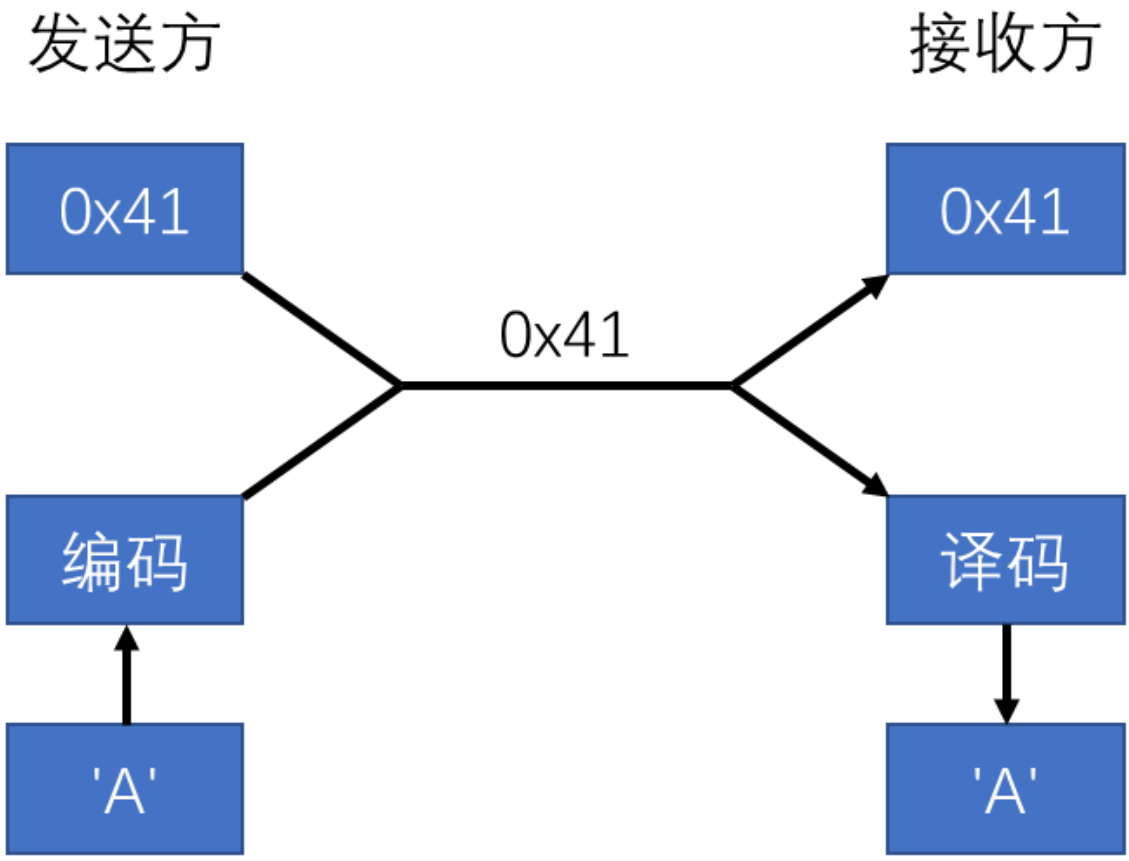
下面是ASCII码表，原始数据编码为字符数据，或者字符数据要译码为原始数据都需要按照这个码表来进行对应。

| DEC | HEX | 字符 | 解释 |
|---|---|---|---|
| 0 | 0x00 | (NUL) | 空字符 |
| 1 | 0x01 | (SOH) | 标题开始 |
| 2 | 0x02 | (STX) | 正文开始 |
| 3 | 0x03 | (ETX) | 正文结束 |
| 4 | 0x04 | (EOT) | 传输结束 |
| 5 | 0x05 | (ENQ) | 请求 |
| 6 | 0x06 | (ACK) | 收到通知 |
| 7 | 0x07 | (BEL) | 响铃 (\a) |
| 8 | 0x08 | (BS) | 退格 (\b) |
| 9 | 0x09 | (HT) | 水平制表符 (\t) |
| 10 | 0x0A | (LF) | 换行键 (\n) |
| 11 | 0x0B | (VT) | 垂直制表符 (\v) |
| 12 | 0x0C | (FF) | 换页键 (\f) |
| 13 | 0x0D | (CR) | 回车键 (\r) |
| 14 | 0x0E | (SO) | 不用切换 |
| 15 | 0x0F | (SI) | 启用切换 |
| 16 | 0x10 | (DLE) | 数据链路转义 |
| 17 | 0x11 | (DC1) | 设备控制1 |
| 18 | 0x12 | (DC2) | 设备控制2 |
| 19 | 0x13 | (DC3) | 设备控制3 |
| 20 | 0x14 | (DC4) | 设备控制4 |
| 21 | 0x15 | (NAK) | 拒绝接收 |
| 22 | 0x16 | (SYN) | 同步空闲 |
| 23 | 0x17 | (ETB) | 结束传输块 |
| 24 | 0x18 | (CAN) | 取消 |
| 25 | 0x19 | (EM) | 媒介结束 |
| 26 | 0x1A | (SUB) | 代替 |
| 27 | 0x1B | (ESC) | 换码 |
| 28 | 0x1C | (FS) | 文件分隔符 |
| 29 | 0x1D | (GS) | 分组符 |
| 30 | 0x1E | (RS) | 记录分隔符 |
| 31 | 0x1F | (US) | 单元分隔符 |

| DEC | HEX | 字符 | 解释 |
|---|---|---|---|
| 32 | 0x20 | (space) | 空格 |
| 33 | 0x21 | ! | 叹号 |
| 34 | 0x22 | " | 双引号 |
| 35 | 0x23 | # | 井号 |
| 36 | 0x24 | $ | 美元符 |
| 37 | 0x25 | % | 百分号 |
| 38 | 0x26 | & | 与号 |
| 39 | 0x27 | ' | 单引号 |
| 40 | 0x28 | ( | 左括号 |
| 41 | 0x29 | ) | 右括号 |
| 42 | 0x2A | * | 星号 |
| 43 | 0x2B | + | 加号 |
| 44 | 0x2C | , | 逗号 |
| 45 | 0x2D | - | 减号 |
| 46 | 0x2E | . | 句号 |
| 47 | 0x2F | / | 斜杠 |
| 48 | 0x30 | 0 | 字符0 |
| 49 | 0x31 | 1 | 字符1 |
| 50 | 0x32 | 2 | 字符2 |
| 51 | 0x33 | 3 | 字符3 |
| 52 | 0x34 | 4 | 字符4 |
| 53 | 0x35 | 5 | 字符5 |
| 54 | 0x36 | 6 | 字符6 |
| 55 | 0x37 | 7 | 字符7 |
| 56 | 0x38 | 8 | 字符8 |
| 57 | 0x39 | 9 | 字符9 |
| 58 | 0x3A | : | 冒号 |
| 59 | 0x3B | ; | 分号 |
| 60 | 0x3C | < | 小于 |
| 61 | 0x3D | = | 等号 |
| 62 | 0x3E | > | 大于 |
| 63 | 0x3F | ? | 问号 |

| DEC | HEX | 字符 | 解释 |
|---|---|---|---|
| 64 | 0x40 | @ | 电子邮件符号 |
| 65 | 0x41 | A | 大写字母A |
| 66 | 0x42 | B | 大写字母B |
| 67 | 0x43 | C | 大写字母C |
| 68 | 0x44 | D | 大写字母D |
| 69 | 0x45 | E | 大写字母E |
| 70 | 0x46 | F | 大写字母F |
| 71 | 0x47 | G | 大写字母G |
| 72 | 0x48 | H | 大写字母H |
| 73 | 0x49 | I | 大写字母I |
| 74 | 0x4A | J | 大写字母J |
| 75 | 0x4B | K | 大写字母K |
| 76 | 0x4C | L | 大写字母L |
| 77 | 0x4D | M | 大写字母M |
| 78 | 0x4E | N | 大写字母N |
| 79 | 0x4F | O | 大写字母O |
| 80 | 0x50 | P | 大写字母P |
| 81 | 0x51 | Q | 大写字母Q |
| 82 | 0x52 | R | 大写字母R |
| 83 | 0x53 | S | 大写字母S |
| 84 | 0x54 | T | 大写字母T |
| 85 | 0x55 | U | 大写字母U |
| 86 | 0x56 | V | 大写字母V |
| 87 | 0x57 | W | 大写字母W |
| 88 | 0x58 | X | 大写字母X |
| 89 | 0x59 | Y | 大写字母Y |
| 90 | 0x5A | Z | 大写字母Z |
| 91 | 0x5B | [ | 左方括号 |
| 92 | 0x5C | \ | 反斜杠 |
| 93 | 0x5D | ] | 右方括号 |
| 94 | 0x5E | ^ | 次方号 |
| 95 | 0x5F | _ | 下划线 |

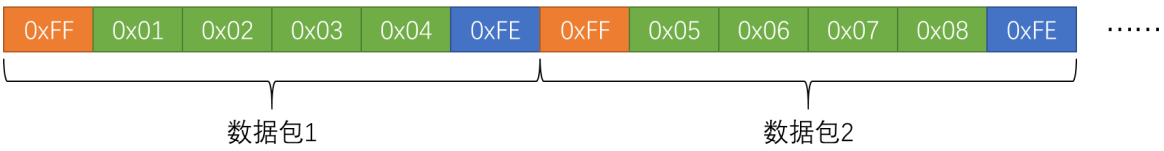
| DEC | HEX | 字符 | 解释 |
|---|---|---|---|
| 96 | 0x60 | ` | 反单引号 |
| 97 | 0x61 | a | 小写字母a |
| 98 | 0x62 | b | 小写字母b |
| 99 | 0x63 | c | 小写字母c |
| 100 | 0x64 | d | 小写字母d |
| 101 | 0x65 | e | 小写字母e |
| 102 | 0x66 | f | 小写字母f |
| 103 | 0x67 | g | 小写字母g |
| 104 | 0x68 | h | 小写字母h |
| 105 | 0x69 | i | 小写字母i |
| 106 | 0x6A | j | 小写字母j |
| 107 | 0x6B | k | 小写字母k |
| 108 | 0x6C | l | 小写字母l |
| 109 | 0x6D | m | 小写字母m |
| 110 | 0x6E | n | 小写字母n |
| 111 | 0x6F | o | 小写字母o |
| 112 | 0x70 | p | 小写字母p |
| 113 | 0x71 | q | 小写字母q |
| 114 | 0x72 | r | 小写字母r |
| 115 | 0x73 | s | 小写字母s |
| 116 | 0x74 | t | 小写字母t |
| 117 | 0x75 | u | 小写字母u |
| 118 | 0x76 | v | 小写字母v |
| 119 | 0x77 | w | 小写字母w |
| 120 | 0x78 | x | 小写字母x |
| 121 | 0x79 | y | 小写字母y |
| 122 | 0x7A | z | 小写字母z |
| 123 | 0x7B | { | 左花括号 |
| 124 | 0x7C | \| | 竖线 |
| 125 | 0x7D | } | 右花括号 |
| 126 | 0x7E | ~ | 波浪号 |
| 127 | 0x7F | (DEL) | 删除 |



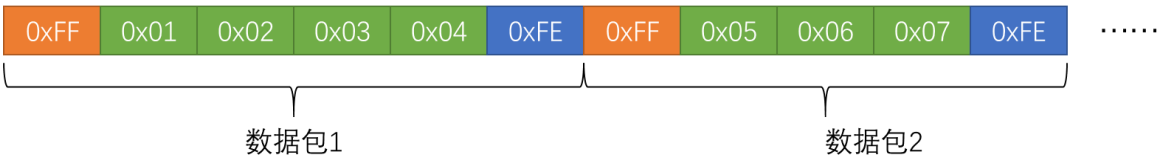

2、Hex数据包

串口数据包通常使用的是额外添加包头、包尾这种方式。比如：下面这里就是两种数据包格式。

（1）固定包长，含包头包尾




（2）可变包长，含包头包尾



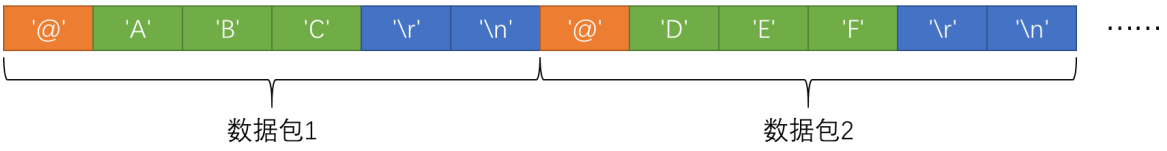

这里有几个问题需要看看。第一个问题，就是包头包尾和数据载荷重复的问题。比如这里定义0xFF为包头，0xFE为包尾。如果传输的数据本身就是0xFF和0xFE，该怎么办呢？此时就会存在数据和包头、包尾重复，这样就可能会引起误判。对于这个问题有如下几种解决方法：第一种限制载荷数据的范围。条件允许的情况下可以对要发送的数据进行限幅。比如，陀螺仪的X、Y、Z轴的三个数据变化范围都可以限制为是0-100，那就们可以在载荷中只发送0-100的数据，这样就不会和包头、包尾重复了。第二种如果无法避免载荷数据和包头包尾重复，那么就尽量使用固定长度的数据包。这样由于载荷数据是固定的，只要通过包头、包尾对齐了数据，就可以准确知道哪个数据应该是包头、包尾，哪个数据应该是载荷数据。在接收载荷数据的时候，并不会判断它是否是包头、包尾，而在接收包头、包尾的时候，也不用判断它是不是确实是包头、包尾了。第三种就是增加包头、包尾的数量，并且让它尽量呈现出载荷数据出现不了的状态。比如，使用0xFFFE作为包头，0xFDFC作为包尾，这样也可以避免载荷数据和包头包尾重复的情况发生。第二个问题，这个包头包尾并不是全部都需要的。比如，可以只要一个包头，把包尾删掉。这样数据包的格式就是一个包头0xFF加4个数据，这样也是可以的。当检测到0xFF开始接收，收够4个字节后，置标记位，对一个数据包接收完成，这样也可以。不过这样载荷和包头重复的问题会更严

重一些。比如最严重的情况下，载荷全是0xFF，包头也是0xFF，那就无法知道哪个是包头了。而加上0xFE作为包尾，无论数据怎么变化，都是可以分辨出包头、包尾的。第三个问题，就是固定包长和可变包长的选择问题。对应HEX数据包来说，如果载荷会出现和包头包尾重复的情况，那就最好选择固定包长，这样可以避免接收错误。如果又会重复又选择可变包长，那数据很容易就乱套了。如果载荷不会和包头包尾重复，那可以选择可变包长。数据长度像4位、3位、1位、10位这样来回任意变肯定都没问题，因为包头包尾是唯一的。只要出现包头就开始数据包，只要出现包尾就结束数据包，这样就非常灵活了。最后一个问题，就是各种数据转换为字节流的问题。数据包都是一个字节一个字节组成的，如果想发送16位的整型数据，32位的整型数据，float，double，甚至是结构体，其实都没问题。因为它们内部其实都是由一个字节一个字节组成的，只需要用一个uint8_t的指针指向它，把它们当做一个字节数组发送就行了。

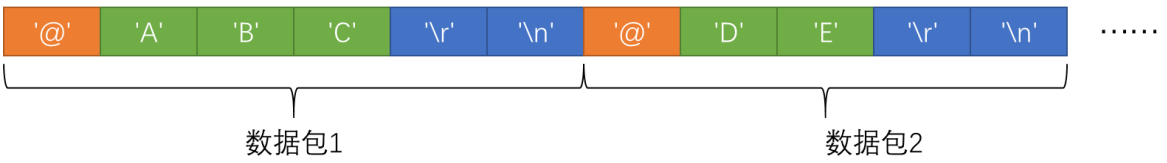
3、文本数据包

在HEX数据包里面，数据都是以原始的字节数据本身呈现的。而在文本数据包里面，每个字节就经过了一层编码和译码，最终表现出来的就是文本格式。但实际上，每个文本字符背后，其实都还是一个字节的HEX数据。

（1）固定包长，含包头包尾




（2）可变包长，含包头包尾




由于数据译码成了字符形式，这就会存在大量的字符可以作为包头包尾，可以有效避免载荷和包头、包尾重复的问题。比如，规定以@这个字符作为包头，以"\r\n"这个换行符作为包尾。在载荷数据中间可以出现除了包头、包尾的任意字符，这是很容易做到的。所以文本数据包基本不用担心载荷和包头、包尾重复的问题，使用非常灵活。可变包长，各种字母、符号、数字都可以随意使用，当接收到载荷数据之后，得到的就是一个字符串。在软件中再对字符串进行操作和判断，就可以实现各种指令控制的功能了。而且字符串数据包表达的意义很明显，可以把字符串数据包直接打印到PC端的串口助手上，什么指令，什么数据一眼就能看明白。所以这个文本数据包通常会以换行作为包尾，这样在打印的时候，就可以一行一行的显示了，非常方便。

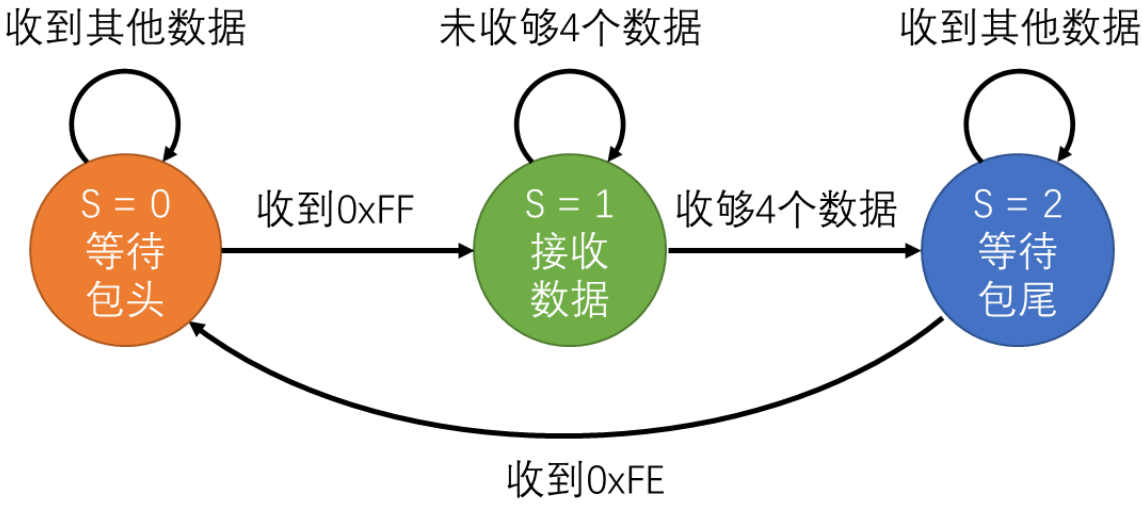
总结一下：

HEX数据包优点是传输最直接，解析数据非常简单，比较适合一些模块发送原始的数据。比如，一些使用串口通信的陀螺仪，温湿度传感器等。缺点就是灵活性不足，载荷容易和包头、包尾重复。
文本数据包优点是数据直观易理解，非常灵活，比较适合一些输入指令进行人机交互的场合。比如，蓝牙模块常用的AT指令，CNC和3D打印机常用的G代码，都是文本数据包的格式。缺点就是解析效率低。比如，发送一个数100，HEX数据包就是一个字节100就完了。文本数据包就得是三个字节的字符100，收到之后还要把字符转换成数据才能得到100。

故而工作中需要根据实际场景来选择和设计数据包的格式。

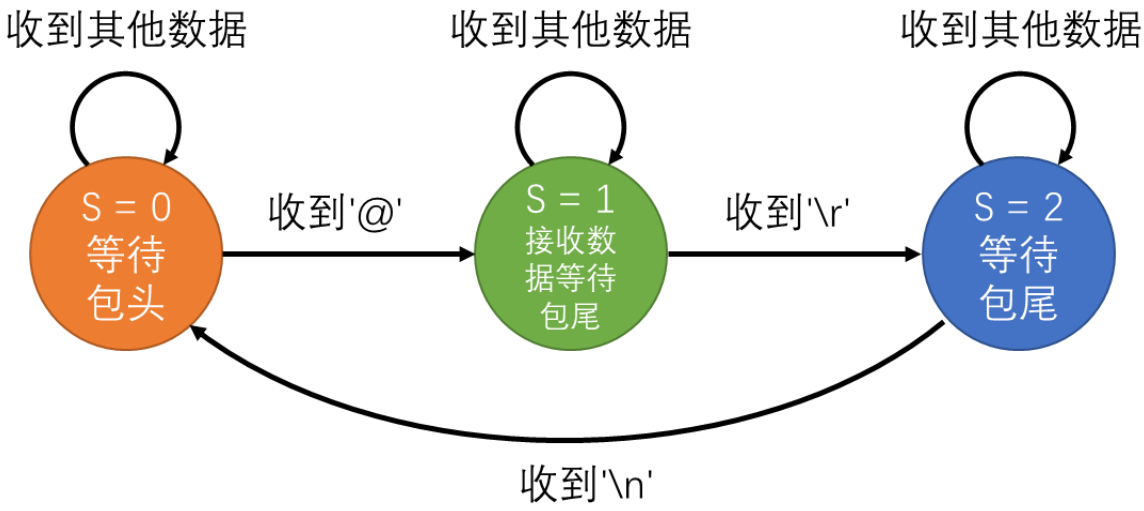
4、Hex数据包接收






5、文本数据包接收



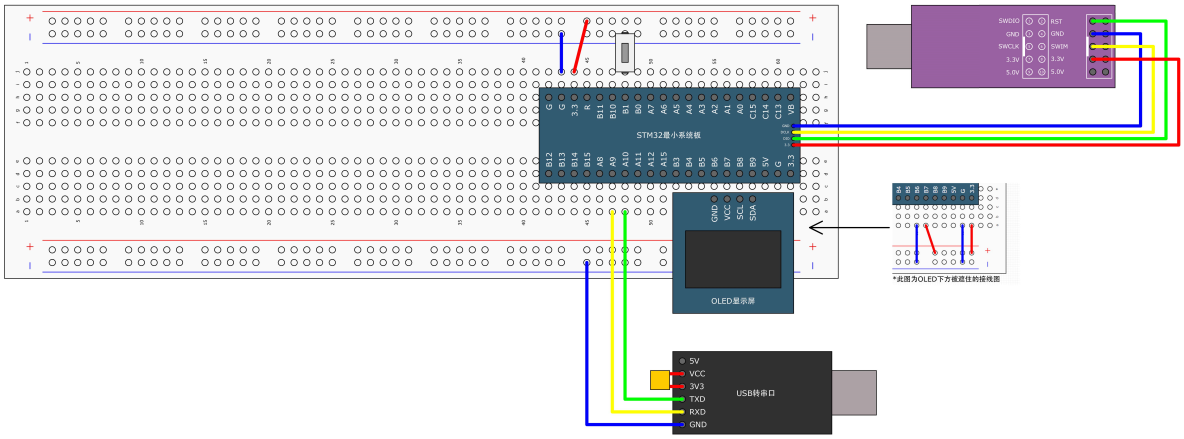

## 五、串口数据包实验

1、串口收发Hex数据包

（1）实验目的

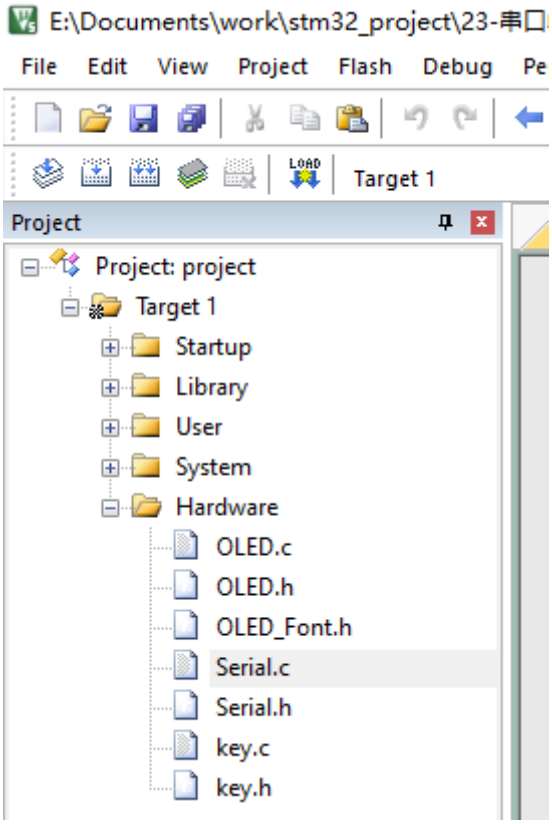
通过控制STM32的USART1实现与PC通信，按键即可发送HEX数据到PC端的串口工具，也可以接收PC利用串口工具发给STM32的HEX数据，将发送和接收的数据分别也显示在OLED上。

（2）硬件电路接线图




（3）程序设计

工程结构如下图：




其中Hardware组与之前工程不一样，这个组主要Serial相关驱动代码增加了中断处理Hex数据包的函数，此外按键驱动还是以前使用过的，这里直接复制过来了。具体代码展示如下：

Serial.c

```
#include "stm32f10x.h"                  // Device header
#include <stdio.h>
#include <stdarg.h>

uint8_t Serial_TxPacket[4];     // 发送数据包数组，数据包格式：FF 01 02 03 04 FE
uint8_t Serial_RxPacket[4];     // 接收数据包数组
uint8_t Serial_RxFlag;      // 接收数据包标志位

void Serial_Init(void) {
    RCC_APB2PeriphClockCmd(RCC_APB2Periph_USART1, ENABLE);      // 打开USART1的时钟
    RCC_APB2PeriphClockCmd(RCC_APB2Periph_GPIOA, ENABLE);       // 打开GPIOA的时钟
```

```
    GPIO_InitTypeDef GPIO_InitStructure;
    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_AF_PP;     // 配置复用推挽输出
    GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
    GPIO_InitStructure.GPIO_Pin = GPIO_Pin_9;
    GPIO_Init(GPIOA, &GPIO_InitStructure);      // 初始化GPIOA

    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_IPU;       // 配置上拉输入
    GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
    GPIO_InitStructure.GPIO_Pin = GPIO_Pin_10;
    GPIO_Init(GPIOA, &GPIO_InitStructure);      // 初始化GPIOA

    USART_InitTypeDef USART_InitStructure;
    USART_InitStructure.USART_BaudRate = 9600;
    USART_InitStructure.USART_HardwareFlowControl = USART_HardwareFlowControl_None;
// 配置不使用流控
    USART_InitStructure.USART_Mode = USART_Mode_Tx | USART_Mode_Rx;
    USART_InitStructure.USART_Parity = USART_Parity_No;
    USART_InitStructure.USART_StopBits = USART_StopBits_1;
    USART_InitStructure.USART_WordLength = USART_WordLength_8b;
    USART_Init(USART1, &USART_InitStructure);       // 初始化USART1

    USART_ITConfig(USART1, USART_IT_RXNE, ENABLE); // 配置串口中断方式

    NVIC_PriorityGroupConfig(NVIC_PriorityGroup_2);

    NVIC_InitTypeDef NVIC_InitStructure;
    NVIC_InitStructure.NVIC_IRQChannel = USART1_IRQn;
    NVIC_InitStructure.NVIC_IRQChannelCmd = ENABLE;
    NVIC_InitStructure.NVIC_IRQChannelPreemptionPriority = 1;
    NVIC_InitStructure.NVIC_IRQChannelSubPriority = 1;
    NVIC_Init(&NVIC_InitStructure);             // 初始化NVIC

    // USART使能
    USART_Cmd(USART1, ENABLE);
}

void Serial_SendByte(uint8_t Byte) {
    USART_SendData(USART1, Byte);
    while (USART_GetFlagStatus(USART1, USART_FLAG_TXE) == RESET);
}

void Serial_SendArray(uint8_t *Array, uint16_t length) {
    uint16_t i;
    for (i = 0; i < length; i++) {
        Serial_SendByte(Array[i]);
    }
}

void Serial_SendString(char *String) {
    uint16_t i;
```

```
    for (i = 0; String[i] != '\0'; i++) {
        Serial_SendByte(String[i]);
    }
}

uint32_t Serial_Pow(uint32_t X, uint32_t Y) {
    uint32_t Result = 1;
    while (Y--) {
        Result *= X;
    }
    return Result;
}

void Serial_SendNumber(uint32_t Number, uint16_t length) {
    uint16_t i;
    for (i = 0; i < length; i++) {
        Serial_SendByte(Number / Serial_Pow(10, length - i - 1) % 10 + '0');
    }
}

// 重写printf底层的fputc，重定向输出到串口
int fputc(int ch, FILE *f) {
    Serial_SendByte(ch);
    return ch;
}

void Serial_Printf(char *format, ...) {
    char String[100];
    va_list arg;
    va_start(arg, format);
    vsprintf(String, format, arg);
    va_end(arg);
    Serial_SendString(String);
}

void Serial_SendPacket(void) {
    // Serial_TxPacket数组的内容将加上包头（FF）包尾（FE）后，作为数据包发送出去
    Serial_SendByte(0xff);
    Serial_SendArray(Serial_TxPacket, 4);
    Serial_SendByte(0xfe);
}

uint8_t Serial_GetRxFlag(void) {
    if (Serial_RxFlag == 1) {
        Serial_RxFlag = 0;
        return 1;
    }
    return 0;
}

void USART1_IRQHandler(void) {
```

```
    static uint8_t RxState = 0;     // 定义表示当前状态机状态的静态变量
    static uint8_t pRxPacket = 0;   // 定义表示当前接收数据位置的静态变量
    // 判断是否是USART1的接收事件触发的中断
    if (USART_GetITStatus(USART1, USART_IT_RXNE) == SET) {
        uint8_t RxData = USART_ReceiveData(USART1);     // 读取数据寄存器，存放在接收的数据
变量
        // 当前状态为0，接收数据包包头
        if (RxState == 0) {
            // 检查包头
            if (RxData == 0xFF) {
                RxState = 1;    // 置下一个状态
                pRxPacket = 0;  // 数据包的位置归零
            }
        // 当前状态为1，接收数据包数据
        } else if (RxState == 1) {
            Serial_RxPacket[pRxPacket] = RxData;    // 将数据存入数据包数组的指定位置
            pRxPacket++;    // 数据包的位置自增
            // 检查是否收够了4个数据
            if (pRxPacket >= 4) {
                RxState = 2;    // 置下一个状态
            }
        // 当前状态为2，接收数据包包尾
        } else if (RxState == 2) {
            // 检查包尾
            if (RxData == 0xFE) {
                RxState = 0;    // 状态归0
                Serial_RxFlag = 1;  // 接收数据包标志位置1，成功接收一个数据包
            }
        }
        USART_ClearITPendingBit(USART1, USART_IT_RXNE);     // 清除标志位
    }
}
```

Serial.h

```
#ifndef __SERIAL_H
#define __SERIAL_H

#include <stdio.h>

extern uint8_t Serial_TxPacket[];
extern uint8_t Serial_RxPacket[];

void Serial_Init(void);
void Serial_SendByte(uint16_t Byte);
void Serial_SendArray(uint8_t *Array, uint16_t length);
void Serial_SendString(char *String);
void Serial_SendNumber(uint32_t Number, uint16_t length);
void Serial_Printf(char *format, ...);
void Serial_SendPacket(void);
```

```
uint8_t Serial_GetRxFlag(void);

#endif
```

key.c

```
#include "stm32f10x.h"                  // Device header
#include "Delay.h"

void KEY_Init(void) {
    RCC_APB2PeriphClockCmd(RCC_APB2Periph_GPIOB, ENABLE);
    GPIO_InitTypeDef GPIO_InitStructure;
    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_IPU;
    GPIO_InitStructure.GPIO_Pin = GPIO_Pin_1 | GPIO_Pin_11;
    GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;

    GPIO_Init(GPIOB, &GPIO_InitStructure);
}

uint8_t KEY_GetNum(void) {
    uint8_t KeyNum = 0;
    if (GPIO_ReadInputDataBit(GPIOB, GPIO_Pin_1) == 0) {
        Delay_ms(20); // 按键后防抖
        while (GPIO_ReadInputDataBit(GPIOB, GPIO_Pin_1) == 0); // 等待按键抬起
        Delay_ms(20); // 按键抬起后防抖
        KeyNum = 1;
    }
    if (GPIO_ReadInputDataBit(GPIOB, GPIO_Pin_11) == 0) {
        Delay_ms(20);
        while (GPIO_ReadInputDataBit(GPIOB, GPIO_Pin_11) == 0);
        Delay_ms(20);
        KeyNum = 2;
    }
    return KeyNum;
}
```

key.h

```
# ifndef __KEY_H
# define __KEY_H

void KEY_Init(void);
uint8_t KEY_GetNum(void);

# endif
```

main.c

```c
#include "stm32f10x.h"                  // Device header
#include "OLED.h"
#include "key.h"
#include "Serial.h"

uint8_t KeyNum;     // 用于接收按键键码的变量

int main(void){

    OLED_Init();
    KEY_Init();
    Serial_Init();

    OLED_ShowString(1, 1, "TxPacket:");
    OLED_ShowString(3, 1, "RxPacket:");

    // 设置发送数据包数组的初始值，用于测试
    Serial_TxPacket[0] = 0x01;
    Serial_TxPacket[1] = 0x02;
    Serial_TxPacket[2] = 0x03;
    Serial_TxPacket[3] = 0x04;

    while(1){
        KeyNum = KEY_GetNum();
        if (KeyNum == 1) {
            // 测试数据自增
            Serial_TxPacket[0]++;
            Serial_TxPacket[1]++;
            Serial_TxPacket[2]++;
            Serial_TxPacket[3]++;

            // 串口发送数据包Serial_TxPacket
            Serial_SendPacket();

            OLED_ShowHexNum(2, 1, Serial_TxPacket[0], 2);
            OLED_ShowHexNum(2, 4, Serial_TxPacket[1], 2);
            OLED_ShowHexNum(2, 7, Serial_TxPacket[2], 2);
            OLED_ShowHexNum(2, 10, Serial_TxPacket[3], 2);
        }
        // 检查是否接收到数据包
        if (Serial_GetRxFlag() == 1) {
            OLED_ShowHexNum(4, 1, Serial_RxPacket[0], 2);
            OLED_ShowHexNum(4, 4, Serial_RxPacket[1], 2);
            OLED_ShowHexNum(4, 7, Serial_RxPacket[2], 2);
            OLED_ShowHexNum(4, 10, Serial_RxPacket[3], 2);
        }
    }
}
```

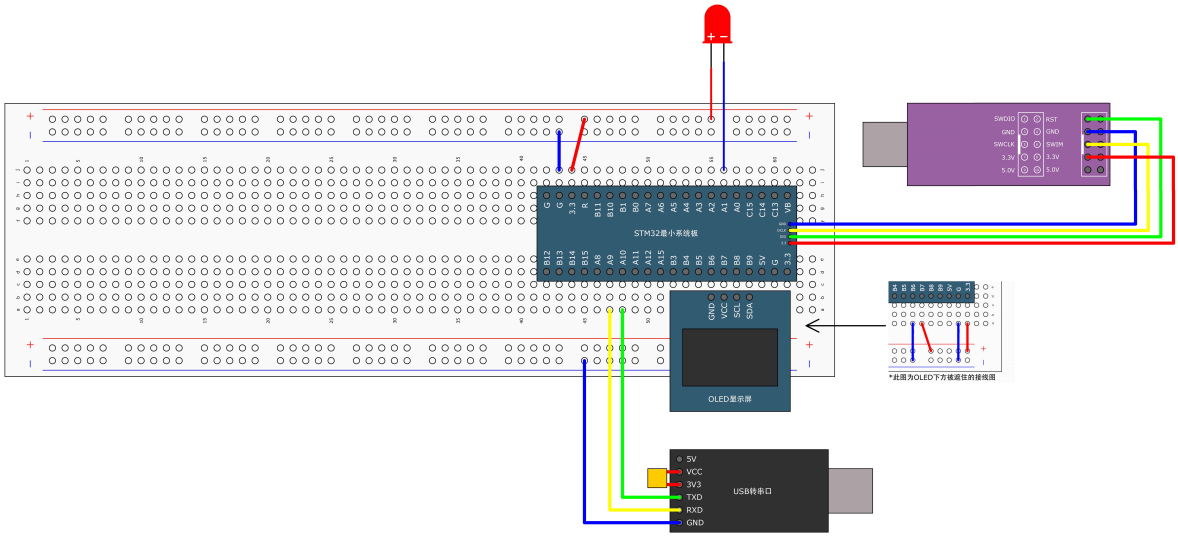
（4）代码完成后，编译并下载到硬件中执行，会发现实验结果是通过按键控制串口发送或接收PC端的串口工具发送的HEX数据，同时在STM32外接的OLED显示数据会对应变化，符合预期设计。

2、串口收发文本数据包

（1）实验目的

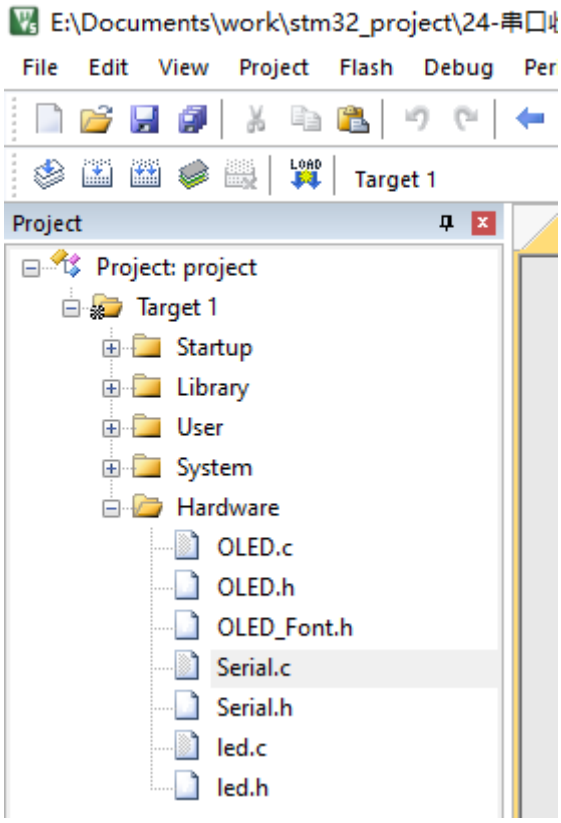
通过控制STM32的USART1实现与PC通信，利用PC端的串口工具给STM32发送字符串指令，从而控制LED的亮灭状态，同时分别在OLED上显示接收和回传的文本数据。

（2）硬件电路接线图




（3）程序设计

工程结构如下图：




其中Hardware组与之前工程相比，这个组主要Serial相关驱动代码增加了中断处理文本数据包的函数，此外LED驱动还是以前使用过的，这里直接复制过来了。具体代码展示如下：

Serial.c

```c
#include "stm32f10x.h"                  // Device header
#include <stdio.h>
#include <stdarg.h>

char Serial_RxPacket[100];
uint8_t Serial_RxFlag;

void Serial_Init(void) {
    RCC_APB2PeriphClockCmd(RCC_APB2Periph_USART1, ENABLE);      // 打开USART1的时钟
    RCC_APB2PeriphClockCmd(RCC_APB2Periph_GPIOA, ENABLE);       // 打开GPIOA的时钟

    GPIO_InitTypeDef GPIO_InitStructure;
    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_AF_PP;     // 配置复用推挽输出
    GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
    GPIO_InitStructure.GPIO_Pin = GPIO_Pin_9;
    GPIO_Init(GPIOA, &GPIO_InitStructure);      // 初始化GPIOA

    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_IPU;       // 配置上拉输入
    GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
    GPIO_InitStructure.GPIO_Pin = GPIO_Pin_10;
    GPIO_Init(GPIOA, &GPIO_InitStructure);      // 初始化GPIOA

    USART_InitTypeDef USART_InitStructure;
    USART_InitStructure.USART_BaudRate = 9600;
    USART_InitStructure.USART_HardwareFlowControl = USART_HardwareFlowControl_None;
// 配置不使用流控
    USART_InitStructure.USART_Mode = USART_Mode_Tx | USART_Mode_Rx;
    USART_InitStructure.USART_Parity = USART_Parity_No;
    USART_InitStructure.USART_StopBits = USART_StopBits_1;
    USART_InitStructure.USART_WordLength = USART_WordLength_8b;
    USART_Init(USART1, &USART_InitStructure);       // 初始化USART

    USART_ITConfig(USART1, USART_IT_RXNE, ENABLE); // 配置串口中断方式

    NVIC_PriorityGroupConfig(NVIC_PriorityGroup_2);

    NVIC_InitTypeDef NVIC_InitStructure;
    NVIC_InitStructure.NVIC_IRQChannel = USART1_IRQn;
    NVIC_InitStructure.NVIC_IRQChannelCmd = ENABLE;
    NVIC_InitStructure.NVIC_IRQChannelPreemptionPriority = 1;
    NVIC_InitStructure.NVIC_IRQChannelSubPriority = 1;
    NVIC_Init(&NVIC_InitStructure);     // 初始化NVIC

    // USART使能
    USART_Cmd(USART1, ENABLE);
}

void Serial_SendByte(uint8_t Byte) {
    USART_SendData(USART1, Byte);
    while (USART_GetFlagStatus(USART1, USART_FLAG_TXE) == RESET);
}
```

```c
void Serial_SendArray(uint8_t *Array, uint16_t length) {
    uint16_t i;
    for (i = 0; i < length; i++) {
        Serial_SendByte(Array[i]);
    }
}

void Serial_SendString(char *String) {
    uint16_t i;
    for (i = 0; String[i] != '\0'; i++) {
        Serial_SendByte(String[i]);
    }
}

uint32_t Serial_Pow(uint32_t X, uint32_t Y) {
    uint32_t Result = 1;
    while (Y--) {
        Result *= X;
    }
    return Result;
}

void Serial_SendNumber(uint32_t Number, uint16_t length) {
    uint16_t i;
    for (i = 0; i < length; i++) {
        Serial_SendByte(Number / Serial_Pow(10, length - i - 1) % 10 + '0');
    }
}

// 重写printf底层的fputc，重定向输出到串口
int fputc(int ch, FILE *f) {
    Serial_SendByte(ch);
    return ch;
}

void Serial_Printf(char *format, ...) {
    char String[100];
    va_list arg;
    va_start(arg, format);
    vsprintf(String, format, arg);
    va_end(arg);
    Serial_SendString(String);
}

void USART1_IRQHandler(void) {
    static uint8_t RxState = 0;     // 定义表示当前状态机状态的静态变量
    static uint8_t pRxPacket = 0;   // 定义表示当前接收数据位置的静态变量
    // 判断是否是USART1的接收事件触发的中断
    if (USART_GetITStatus(USART1, USART_IT_RXNE) == SET) {
```

```
        uint8_t RxData = USART_ReceiveData(USART1);      // 读取数据寄存器，存放在接收的数据
变量
        // 当前状态为0，接收数据包包头
        if (RxState == 0) {
            // 如果数据确实是包头，并且上一个数据包已处理完毕
            if (RxData == '@' && Serial_RxFlag == 0) {
                RxState = 1;    // 置下一个状态
                pRxPacket = 0;  // 数据包的位置归零
            }
        // 当前状态为1，接收数据包数据，同时判断是否接收到了第一个包尾
        } else if (RxState == 1) {
            // 检查是否包尾首个字符
            if (RxData == '\r') {
                RxState = 2;    // 置下一个状态
            // 接收到了正常的数据
            } else {
                Serial_RxPacket[pRxPacket] = RxData;    // 将数据存入数据包数组的指定位置
                pRxPacket++;    // 数据包的位置自增
            }
        // 当前状态为2，接收数据包第二个包尾
        } else if (RxState == 2) {
            // 检查包尾
            if (RxData == '\n') {
                RxState = 0;    // 状态归0
                Serial_RxPacket[pRxPacket] = '\0';  // 将收到的字符数据包添加一个字符串结束标
志
                Serial_RxFlag = 1;  // 接收数据包标志位置1，成功接收一个数据包
            }
        }
        USART_ClearITPendingBit(USART1, USART_IT_RXNE);      // 清除标志位
    }
}
```

Serial.h

```
#ifndef __SERIAL_H
#define __SERIAL_H

#include <stdio.h>

extern char Serial_RxPacket[];
extern uint8_t Serial_RxFlag;

void Serial_Init(void);
void Serial_SendByte(uint16_t Byte);
void Serial_SendArray(uint8_t *Array, uint16_t length);
void Serial_SendString(char *String);
void Serial_SendNumber(uint32_t Number, uint16_t length);
void Serial_Printf(char *format, ...);
```

```
#endif
```

led.c

```
#include "stm32f10x.h"                  // Device header

void LED_Init(void) {
    RCC_APB2PeriphClockCmd(RCC_APB2Periph_GPIOA, ENABLE);

    GPIO_InitTypeDef GPIO_InitStructure;
    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_Out_PP;
    GPIO_InitStructure.GPIO_Pin = GPIO_Pin_1 | GPIO_Pin_2;
    GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
    GPIO_Init(GPIOA, &GPIO_InitStructure);

    GPIO_SetBits(GPIOA, GPIO_Pin_1 | GPIO_Pin_2);
}

void LED1_ON(void) {
    GPIO_ResetBits(GPIOA, GPIO_Pin_1);
}

void LED1_OFF(void) {
    GPIO_SetBits(GPIOA, GPIO_Pin_1);
}

void LED1_Turn(void) {
    // pin1输出低电平，灯亮
    if (GPIO_ReadOutputDataBit(GPIOA, GPIO_Pin_1) == 0) {
        GPIO_SetBits(GPIOA, GPIO_Pin_1); // pin1置高电平，灯灭
    } else {
        GPIO_ResetBits(GPIOA, GPIO_Pin_1); // pin1置低电平，灯亮
    }
}

void LED2_ON(void) {
    GPIO_ResetBits(GPIOA, GPIO_Pin_2);
}

void LED2_OFF(void) {
    GPIO_SetBits(GPIOA, GPIO_Pin_2);
}

void LED2_Turn(void) {
    if (GPIO_ReadOutputDataBit(GPIOA, GPIO_Pin_2) == 0) {
        GPIO_SetBits(GPIOA, GPIO_Pin_2);
    } else {
        GPIO_ResetBits(GPIOA, GPIO_Pin_2);
    }
}
```

```
void LED_Set(uint16_t GPIO_Pin, BitAction BitVal) {
    GPIO_WriteBit(GPIOA, GPIO_Pin, BitVal);
}
```

led.h

```
# ifndef __LED_H
# define __LED_H

void LED_Init(void);
void LED1_ON(void);
void LED1_OFF(void);
void LED1_Turn(void);
void LED2_ON(void);
void LED2_OFF(void);
void LED2_Turn(void);
void LED_Set(uint16_t GPIO_Pin, BitAction BitVal);

# endif
```

main.c

```
#include "stm32f10x.h"                  // Device header
#include "OLED.h"
#include "led.h"
#include "Serial.h"
#include <string.h>

int main(void){

    OLED_Init();
    LED_Init();
    Serial_Init();

    OLED_ShowString(1, 1, "TxPacket:");
    OLED_ShowString(3, 1, "RxPacket:");

    while(1){
        // 检查是否接收到数据包
        if (Serial_RxFlag == 1) {
            OLED_ShowString(4, 1, "                ");
            OLED_ShowString(4, 1, Serial_RxPacket);

            // 将收到的数据包与预设的指令进行检查判断
            if (strcmp(Serial_RxPacket, "LED_ON") == 0) {
                LED1_ON();
                Serial_SendString("LED_ON_OK\r\n");     // 向PC回传文本数据包
                OLED_ShowString(2, 1, "                ");
```

```
                OLED_ShowString(2, 1, "LED_ON_OK");
            } else if (strcmp(Serial_RxPacket, "LED_OFF") == 0) {
                LED1_OFF();
                Serial_SendString("LED_ON_OFF\r\n");
                OLED_ShowString(2, 1, "                ");
                OLED_ShowString(2, 1, "LED_ON_OFF");
            } else {
                Serial_SendString("ERROR_COMMAND\r\n");
                OLED_ShowString(2, 1, "                ");
                OLED_ShowString(2, 1, "ERROR_COMMAND");
            }
            Serial_RxFlag = 0;  //处理完成后，需要将接收数据包标志位清零，否则将无法接收后续数
据包
        }
    }
}
```

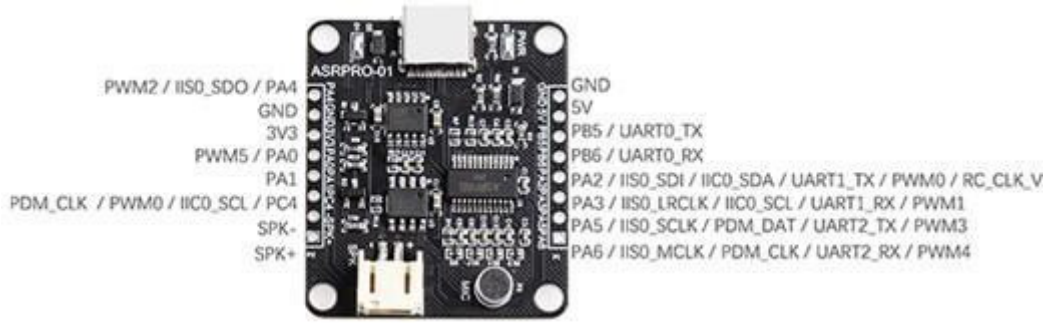
（4）代码完成后，编译并下载到硬件中执行，会发现实验结果是利用PC端的串口工具给STM32发送字符串指令，LED的亮灭状态以及在OLED上显示数据都正常，符合预期设计。

## 六、语音控制：天问ASRPRO-01（开发板5V）

### 1、基本介绍

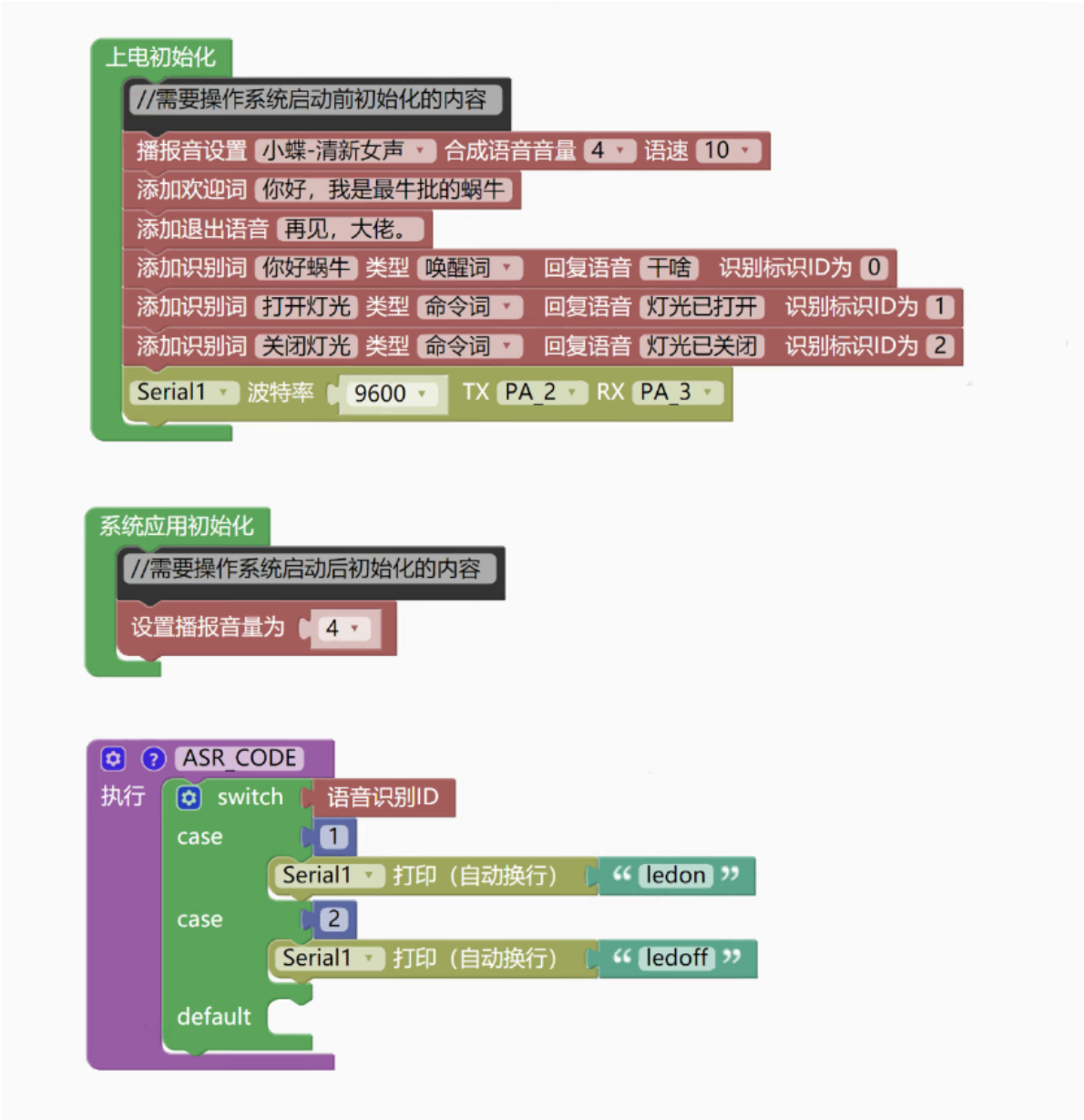
ASRPRO是一款全功能带语音识别的物联网开发板，方便学习，带语音识别的物联网开发板，基于 32 位 RISC-V 内核，内置神经网络处理器，支持 DNNTDN-NRNN 等神经网络及卷积运算，支持语音识别、声纹识别、语音增强、语音检测等功能，具备强劲的回声消除和环境噪声抑制能力。搭配天问 Block 图形化编程软件，快速实现语音交互及控制和各类智能语音物联网方案应用。

长宽为 30x28mm，板载麦克风、指示灯，用户只需要外接喇叭就可以使用，一根 Type-C 线就可以下载程序，并且开发板上有自动断电电路可以实现一键下载，不需要额外的STC-LINK 下载器。

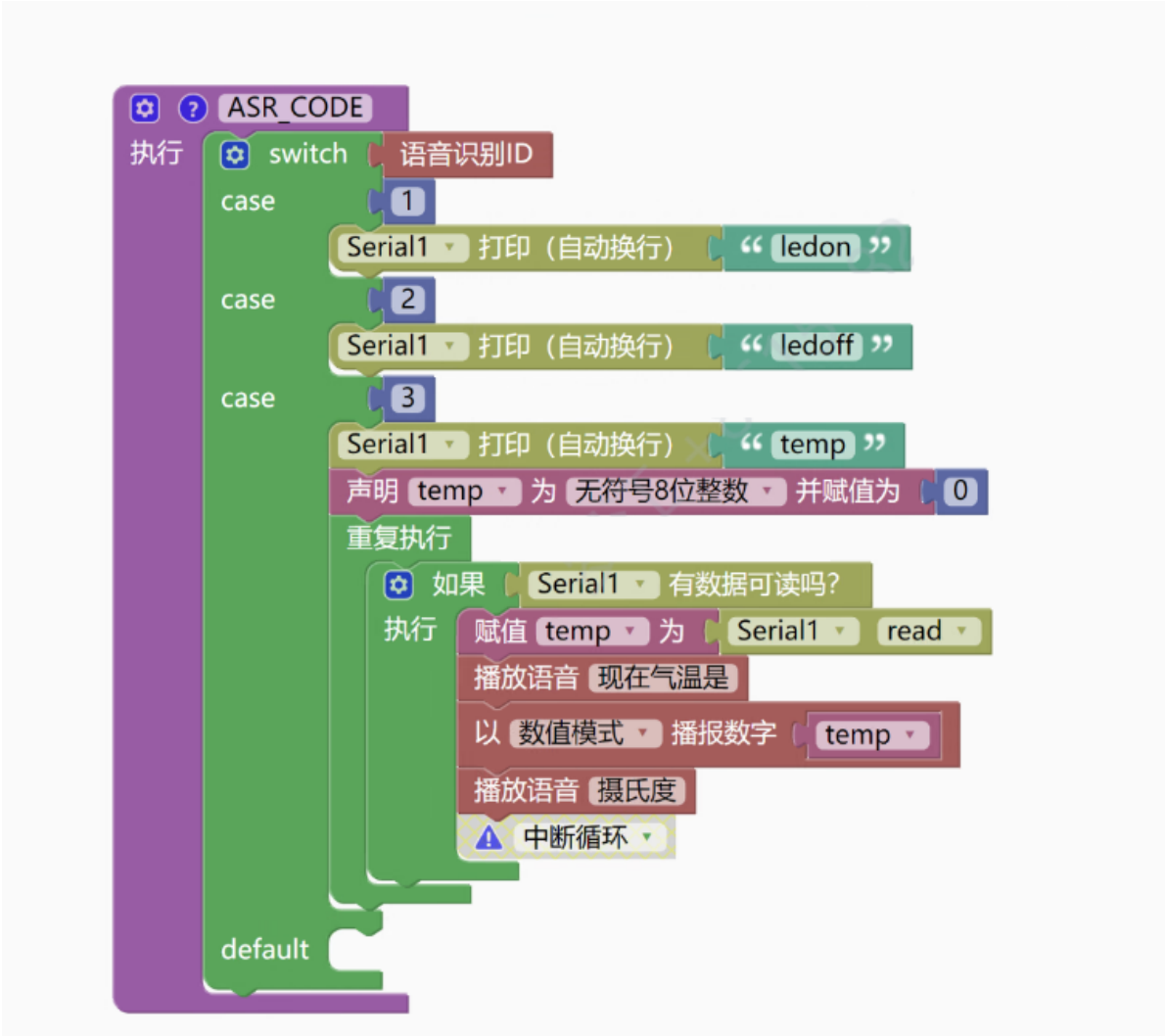



### 2、使用天问Block开发

（1）语音识别开灯（串口发送LEDON），关灯（串口发送LEDOFF）




（2）语音识别温度（接收STM32发的温度信息）




**3、练习**

1. 语音控制风扇开关，快慢。
2. 控制不同灯光的开闭(至少两个)，比如，卧室灯，客厅灯。
3. 控制风扇的开闭和风速:如开风扇，关风扇，一档，二档，三档。
4. 查询当前气温，语音播报当前气温

## 课程小结

1. 串口是一种应用非常广泛的通讯接口，串口成本低，通信线路简单，易于使用，在嵌入式设备领域应用极其广泛。
2. STM32的USART兼具同步和异步通信，此外它还具备多个USART资源，能够与支持众多的外设通讯。
3. 串口通信的主要工作流程是首先初始化串口，进行参数配置，然后发送数据，或者接收数据。在串口通信过程中TX和RX接线很重要，需要避免接错。同时波特率的选择对于通信距离和通信稳定性影响很大。
4. 串口通信中数据包的包长和包尾在不同的数据包模式下有着不一样的要求，特别对于Hex数据包情况下要注意避免包头、包尾与有效数据载荷的重复问题。

## 随堂作业

1. 请谈谈串口通信的机制是怎样的?
2. 请尝试利用串口发送指令控制风扇的转动。